Duale Hochschule Sachsen – Staatliche Studienakademie Leipzig

Gamification in der Programmierausbildung

Ein Lernspiel zur Festigung vorhandener Kenntnisse

Bachelorthesis

zur Erlangung der staatlichen Abschlussbezeichnung eines
Bachelor of Science (B. Sc.)
im Studiengang der Informatik

Modul: 5CS-BSC-60

vorgelegt von:
Max Fritsche
Kurt-Eisner-Str. 46, 04275 Leipzig
Seminargruppe: CS-22-1
Matrikelnummer: 5002242

Erstgutachter:
Prof. Dr. Alexander Paar
Duale Hochschule Sachsen

Zweitgutachter:
Aleksej Tokarev
Security Robotics D&S GmbH

Leipzig, 1. September 2025

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Herausforderungen in der Programmierausbildung 1

1.2 Hypothesen der Arbeit 3

1.3 Aufbau der Arbeit 4

2 Theoretische Grundlagen **5**

2.1 Bedeutung praktischer Übung im Lernprozess 5

2.2 Gamification als didaktische Methode 6

2.3 Gamification versus traditionelle Lernmethoden 9

3 Stand der Technik: Programmierspiele und Lernplattformen **12**

3.1 Blockbasierte Lernumgebungen 12

3.2 Puzzle- und Rätselspiele 13

3.3 Code-Adventure-Spiele 13

3.4 Online-Code-Judge-Plattformen 14

3.5 Fazit: Zusammenfassung und Abgrenzung zum eigenen Ansatz 15

4 Konzeption des Lernspiels **16**

4.1 Zielgruppe und Lernziele 16

4.2 Spielmechanik und Designentscheidungen 17

4.3 Wahl der Technologie 21

5 Implementierung des Lernspiels **25**

5.1 Systemarchitektur 25

5.2 Backend-Implementierung 27

5.3 Frontend-Implementierung (Unity-Client) 33

6 Technische Validierung des Lernspiels **40**

6.1 Testfälle zur Validierung des Online-Code-Judges 40

6.2 Validierung des Movement-Systems 41

6.3 Geplante Studie: Gamifizierter vs. klassischer Lernansatz 42

7 Evaluation des Lernspiels **44**

7.1 Evaluationsansatz und Ziele 44

7.2 Technische Evaluation des Systems 45

7.3 Pädagogisch-didaktische Evaluation 47

7.4 Didaktische Bedeutung der Quest-Struktur 49
7.5 Bestätigung der Hypothese und Fazit der Evaluation 50

8 Fazit und Ausblick **52**
8.1 Fazit 52
8.2 Ausblick 53

Abkürzungsverzeichnis **IV**

Quellenverzeichnis **V**

Anlage 1: Thesenpapier **X**

Anlage 2: Selbständigkeitserklärung **XII**

1 Einleitung

Das Erlernen von Programmierung ist ein komplexer Prozess, der nicht nur das theoretische Verständnis, sondern insbesondere auch die praktische Anwendung der erlernten Konzepte verlangt. Viele Studierende empfinden diesen Prozess als herausfordernd, da es in der traditionellen Programmierausbildung oft an interaktiven und motivierenden Übungsmöglichkeiten mangelt. Insbesondere besteht ein Defizit an gezielten praktischen Übungen, die die Anwendung der theoretischen Konzepte effektiv vertiefen.
Daraus folgt die Notwendigkeit eines neuartigen, spielerischen Ansatzes, der sowohl die Motivation erhöht als auch die praktische Programmiererfahrung der Lernenden stärkt. In dieser Arbeit wird untersucht, inwiefern Gamification dazu beiträgt, die Produktivität und Effektivität in der Programmierausbildung zu steigern.

1.1 Herausforderungen in der Programmierausbildung

Die Ausbildung im Programmieren gilt als anspruchsvoll, insbesondere für Anfänger. Empirische Untersuchungen zeigen, dass ein Großteil der Lernenden Programmieren als schwierig empfindet und Anfänger deutlich mehr Unterstützung benötigen als Fortgeschrittene (SINGH, 2022). Tatsächlich sind in Einsteigerprogrammierkursen hohe Misserfolgs- und Abbruchquoten keine Seltenheit (SINGH, 2022).
Aus Sicht der Studierenden treten zuerst kognitive und fachliche Hürden in den Vordergrund. Wiederkehrend genannt werden Defizite bei Problemlösefähigkeit, Abstraktion und logisch-mathematischem Denken (MEDEIROS u. a., 2019). Problemlösen fungiert dabei zugleich als Voraussetzung und als Lernhürde. Wer Schwierigkeiten hat, Aufgaben systematisch zu analysieren, in Teilprobleme zu zerlegen und eine Lösung zu planen, kämpft in der Folge auch mit der Umsetzung in Code (MEDEIROS u. a., 2019). Zudem ist der Abstraktionsgrad grundlegender Konzepte (Variablen, Datentypen, Speicheradresse, Kontrollstrukturen) hoch, da hierfür oft keine alltagsnahen Analogien existieren. Viele Anfänger verstehen einzelne Konzepte isoliert, scheitern jedoch an der Überführung in funktionsfähige Programme, insbesondere am Schritt vom konzeptuellen zum strategischen Wissen (Planung, Zerlegung, zielgerichtetes Debugging) (MEDEIROS u. a., 2019).
Ein zentrales Hindernis ist außerdem die Syntax. Fehlende oder falsch platzierte Zeichen (z.B. Klammern, Semikolons) und das Missverstehen von Compiler-/Fehlermeldungen führen leicht zu frustrierender Trial-and-Error-Suche statt systematischer Fehlerbehebung

(MEDEIROS u. a., 2019). Die Literatur betont, dass Syntaxfehler als Lernanlass didaktisch adressiert werden sollten, weil geübte Strategien zur Fehlerkorrektur bei Anfängern erst aufgebaut werden müssen (MEDEIROS u. a., 2019). Ebenso wichtig sind Code-Tracing und systematisches Testen, die jedoch von Neulingen oft gemieden oder unsicher ausgeführt werden (MEDEIROS u. a., 2019).

Zu den verhaltensbezogenen Faktoren zählen Motivation, Selbstwirksamkeit/Selbstvertrauen, Zeitmanagement und Lernstrategien. Gerade in frühen Studienphasen wird Programmieren häufig als “schwierig" wahrgenommen. Dies kann Motivations- und Vertrauensverluste nach sich ziehen und eigenständiges Explorieren hemmen. Gleichzeitig berichten Studierende von Zeitknappheit und Schwierigkeiten, außerhalb angeleiteter Sitzungen effektiv zu arbeiten. Sprachliche (Englisch) und mathematische Grundlagen können zusätzliche Hürden darstellen und den Einstieg in Konzepte und Fehlermeldungen erschweren (MEDEIROS u. a., 2019).

Singhs qualitativ angelegte Handlungsforschung mit Studierendenbefragung, Kursreview-Analyse und Dozierendeninterview bestätigt diese Punkte aus einer Praxisperspektive. Ein hoher Anteil der Befragten empfindet Einsteigerprogrammieren als herausfordernd und sieht Mehrbedarf an Unterstützung gegenüber Lernenden mit Vorerfahrung. In Kursreviews erscheinen wiederholt Frustration, Abbruchgedanken und der Wunsch nach besserer Anleitung, insbesondere bei der Aufgabenbearbeitung, Software-Setup und dem Verständnis grundlegender Konzepte (z.B. Variablen, Schleifen). Der Studie zufolge ist die klare Trennung und Verzahnung von syntaktischem, konzeptionellem und strategischem Wissen didaktisch bedeutsam. Letzteres, die Anwendung von Wissen zur realen Problemlösung, stellt den größten Entwicklungssprung dar.

Lehrende sehen sich mit Skalierungs- und Heterogenitätsproblemen konfrontiert. Stark unterschiedliche Vorkenntnisse, große Kohorten, begrenzte Personalressourcen und die Notwendigkeit personalisierten Feedbacks bei gleichzeitig hohem Betreuungsaufwand. Dies erschwert die individuelle Förderung ebenso wie die fortlaufende Diagnose von Fehlkonzepten und Lernständen (MEDEIROS u. a., 2019).

Hinzu kommen methodisch-didaktische Fragen. In der Literatur finden sich zahlreiche Ansätze, etwa aktive Lernformen (Live-Coding, In-Class-Übungen), problemorientiertes Lernen, Peer Instruction bzw. Pair Programming, “Extreme Apprenticeship", spielthematische Aufgaben, Gamification oder Medien- und Visualisierungsansätze. Die Befunde zur Wirksamkeit sind jedoch uneinheitlich und teils widersprüchlich (MEDEIROS u. a., 2019). Einige Studien berichten deutliche Verbesserungen, etwa ein trace-driven teaching approach mit geringeren Abbruch- und Fehlquoten in einer Kohorte. Andere Implementationen zeigen hingegen trotz methodischer Anpassungen keine oder nur gemischte Effekte. Insgesamt fordern die Reviews daher robustere Vergleichsdesigns sowie gemeinsame Benchmarks und Standards für Evaluationen (MEDEIROS u. a., 2019). Ein wiederkehrendes Thema ist der Methoden- und Tool-Mangel gerade vor der eigentlichen Codierung. Für die frühe Phase der Problemformulierung (vom Aufgabenverständnis zur algorithmischen

Skizze) fehlen im Hochschulkontext geeignete, akzeptierte Werkzeuge. Blockbasierte Umgebungen (z.B. aus der Schule) sind zwar hilfreich, werden aber für die Zielgruppe oft als zu “kindlich" wahrgenommen; eine hochschuladäquate Unterstützungslinie ist noch unterentwickelt (MEDEIROS u. a., 2019).
Auch die Sprachwahl (erste Programmiersprache) bleibt ein strategischer Knackpunkt. Einerseits gilt es, Syntaxlast zu vermeiden, die den Blick auf allgemeine Konzepte verstellt; andererseits sollen praxisrelevante Kompetenzen früh aufgebaut werden. Zugleich kann die Wahl einer dominanten “Industriesprache" Fehlwahrnehmungen fördern (etwa, andere Sprachen seien unnötig) und die Fähigkeit “lernen zu lernen" mindern, wenn Lernende zu früh in spezifische Ökosysteme eingebracht werden (MEDEIROS u. a., 2019).
Schließlich sind Curriculum und Reihenfolge der Inhalte offener Forschungsbedarf. Ob man mit strategischen Übersichten beginnt, Syntaxdetails schrittweise “hochzieht" oder konzeptweise durchgeht, direkte Vergleichsstudien zur Reihenfolge der Inhaltsvermittlung fehlen weitgehend. Lehrende müssen daher den Unterricht oft heuristisch gestalten und parallel die Motivation der Studierenden stabilisieren (MEDEIROS u. a., 2019).
Für Einsteigerkurse legt die Literatur nahe, den Fokus konsequent auf (1) Problemformulierung und algorithmisches Denken, (2) den Übergang zu strategischem Wissen (strukturierte Lösungsentwürfe, Test-/Debugging-Strategien) sowie (3) Motivation und Selbstwirksamkeit zu legen und erst danach Sprach-/Framework-Spezifika zu vertiefen. Wo große Kohorten und Heterogenität vorliegen, sollte zeitnahes Feedback priorisiert werden (MEDEIROS u. a., 2019).
In Summe bekräftigen beide Quellen, dass die traditionelle, primär frontale Vermittlung allein nicht reicht. Wirksam sind Lernarrangements, die aktive Auseinandersetzung, sofortiges Feedback, Visualisierung und niedrigschwellige Unterstützung verbinden und die explizit auf den Aufbau von strategischem Programmierwissen zielen. Gamification- und Spielansätze können hier andocken (Motivation, Übungsvolumen, unmittelbare Rückmeldungen), sollten aber konzeptionell so gestaltet sein, dass sie Problemformulierung, Debugging-Strategien und Transfer fördern, nicht nur Punkte sammeln (MEDEIROS u. a., 2019) (SINGH, 2022).

1.2 Hypothesen der Arbeit

Wie sinnvoll Gamification in der Programmierausbildung ist, gilt es in dieser Arbeit rauszufinden. Die zentrale **Hypothese** dieser Arbeit lautet: *“Die Gamification von Programmierausbildung lässt sich in Form eines Zelda-ähnlichen Rollenspiels erfolgreich und wirkungsvoll umsetzen."*
Zur Überprüfung dieser Annahme wird ein funktionsfähiger Prototyp entwickelt, der die didaktischen und technischen Konzepte praktisch erprobt. Ergänzend wird eine empirische Studie skizziert, die die lernmethodischen Grundlagen untersucht und die Wirksamkeit des Gamification-Ansatzes im Kontext der Programmierausbildung reflektiert.

1.3 Aufbau der Arbeit

Die Arbeit beginnt mit einer detaillierten Betrachtung der theoretischen Grundlagen in Bezug auf die Programmierausbildung (Kapitel 2). Anschließend wird der aktuelle Stand der Forschung im Bereich der Programmierspiele analysiert, um bestehende Schwächen zu identifizieren (Kapitel 3). Daraus folgt die Konzeption und Umsetzung eines eigenen Gamification-Ansatzes in Form eines Zelda-artigen Lernspiels, welches auf spezifische Weise praktische Programmierübungen integriert.
Es werden sowohl der verwendete Technologiestack als auch methodische Entscheidungen im Detail erläutert. So wird beispielsweise argumentiert, warum die Umsetzung zunächst mit **C**[1] anstelle von **Python**[2] erfolgt und wie Online-Code-Judges genutzt werden, um die Qualität und Korrektheit der Programmieraufgaben im Spiel automatisch zu bewerten (Kapitel 5).
Darauf aufbauend erfolgt eine technische Validierung des Spiels anhand von konkreten Aufgaben mit exemplarischen Lösungen (korrekten sowie fehlerhaften). Abschließend skizziert die Arbeit ein empirisches Experiment, um die Effektivität der entwickelten Lernmethode zu überprüfen und somit die erste Hypothese zu testen (Kapitel 7).
Das Fazit fasst die Ergebnisse zusammen und reflektiert sowohl die methodischen als auch technischen Aspekte der Arbeit kritisch.

[1] Vgl. ISO/IEC 9899:2018 - Programming Language C, `https://www.iso.org/standard/74528.html`, abgerufen am 30.08.2025.
[2] Vgl. Python Software Foundation: `https://www.python.org/`, abgerufen am 30.08.2025.

2 Theoretische Grundlagen

Im Folgenden werden die theoretischen Grundlagen erläutert, die für die Konzeption des geplanten Lernspiels relevant sind. Darauf aufbauend wird Gamification als didaktische Methode vorgestellt, einschließlich der lernpsychologischen Modelle, die dieses Konzept untermauern und schließlich Gamification im Vergleich zu traditionellen Lernmethoden diskutiert. Ein zentrales Thema ist dabei, wie unmittelbares Feedback und erhöhte Motivation durch Gamification den Lernerfolg insbesondere bei Programmieranfängern fördern.

2.1 Bedeutung praktischer Übung im Lernprozess

Um programmieren zu lernen, reicht passives Wissen nicht aus, aktives Üben ist unverzichtbar. Das Konzept Learning by Doing (Lernen durch Handeln) hat eine lange Tradition seit John Dewey und anderen Vertretern des Konstruktivismus und der Erfahrungslern-Theorie (IFTIKHAR u. a., 2022). Kerngedanke ist, dass Lernen am effektivsten durch eigene Erfahrungen und Handlungen geschieht, anstatt nur durch abstrakte Theorievermittlung (IFTIKHAR u. a., 2022). Lernende sollen mit den Händen (oder im Geist) selbst tätig werden, um Fähigkeiten aufzubauen; dies gilt besonders in praktischen Fächern wie der Programmierung (IFTIKHAR u. a., 2022). Für den Bereich der Programmierung bestätigen verschiedene Studien die herausragende Rolle der Praxis. Theoretisches Wissen, etwa das Lesen über Syntax oder das Verfolgen von Vorlesungen, kann zwar Grundbegriffe vermitteln. Doch um aus diesen Begriffen echte Fertigkeiten zu entwickeln, ist die Anwendung in zahlreichen Übungssituationen erforderlich (IFTIKHAR u. a., 2022). Das Verständnis eines Konzeptes entsteht zunächst durch die Theorie, aber erst das wiederholte Anwenden und Ausprobieren festigt es als Könnerschaft (IFTIKHAR u. a., 2022). So werden Problemlösefähigkeiten typischerweise dadurch ausgebildet, dass man ähnliche Aufgabenstellungen mehrfach praktisch durchdringt und löst (IFTIKHAR u. a., 2022). Insbesondere das Programmieren selbst wird als Fertigkeit angesehen, die nur durch Übung wirklich erlernt werden kann (IFTIKHAR u. a., 2022). Eine Untersuchung fasst prägnant zusammen: „Learning to code is a skill that can be only developed by virtue of practice“, das Programmieren lernt man nur, indem man es immer wieder tut (IFTIKHAR u. a., 2022).

Praktische Übungen bieten dabei mehrere Vorteile. Zum einen fördern sie die Verinnerlichung von Konzepten. Wenn Lernende unmittelbar nach der Einführung eines neuen Konzepts dieses selbst anwenden (z.B. durch Schreiben eines einfachen Programms), wird

das Verständnis vertieft und das Behalten verbessert. Zum anderen entwickeln Studierende durch eigenes Ausprobieren ein besseres Problemverständnis und ein Gefühl für häufige Fallstricke. Gerade im Programmierunterricht berichten Dozenten wie Studierende, dass zusätzliche Übungsaufgaben zu Kernkonzepten (etwa Schleifen oder Datenstrukturen) enorm hilfreich wären, um diese wirklich zu meistern (SINGH, 2022). Wo solche Gelegenheiten fehlen, bleiben Verständnislücken bestehen, die später den Fortschritt hemmen. Nicht zuletzt wirkt sich praktisches Arbeiten positiv auf die Motivation und Eigenständigkeit der Lernenden aus. Wenn Studierende Aufgaben selbst lösen dürfen, werden sie vom passiven Konsumenten zum aktiven Gestalter, Erfolgserlebnisse bei der Lösung stärken das Selbstvertrauen. Dies ist insbesondere in Umgebungen wichtig, in denen keine direkte persönliche Betreuung stattfindet (z.B. beim E-Learning). Studien zeigen, dass in Online-Programmierkursen die Integration eines Learning-by-Doing-Ansatzes sowohl die Vermittlung der Konzepte verbessert als auch die Beteiligung der Studierenden erhöht (IFTIKHAR u. a., 2022). Praktische Online-Labore oder Coding-Übungsplattformen ermöglichen es den Lernenden kontinuierlich zu üben und im eigenen Tempo zu lernen, was zu höherer Engagement-Rate und geringerer Abbruchquote führt (IFTIKHAR u. a., 2022). Allgemein gilt, dass in Fächern, in denen Fähigkeiten und nicht nur Fakten gelernt werden, muss ausreichend geübt werden. Reines Lesen oder Zuhören reicht hier nicht aus (IFTIKHAR u. a., 2022). Daher sollte ein didaktisches Konzept für Programmieranfänger unbedingt viele und gut strukturierte Übungsmöglichkeiten vorsehen.
Allerdings entfaltet die praktische Übung ihre volle Wirkung erst in Kombination mit Feedback. Ohne Rückmeldung wissen Lernende nicht, ob ihre Lösung korrekt ist oder wo genau Fehler liegen. Hier kommt der nächste Baustein ins Spiel. Gamification als Methode, die kontinuierliche Übung und sofortiges Feedback auf motivierende Weise verbindet.

2.2 Gamification als didaktische Methode

Gamification bezeichnet den Einsatz von Spielelementen in einem eigentlich spielfremden Kontext, hier der Bildung, mit dem Ziel, die Motivation und das Engagement der Lernenden zu steigern (SMIDERLE u. a., 2020). Anders als beim reinen Spiel bleibt der eigentliche Zweck das Lernen; durch die Integration von Game-Design-Elementen wie Punkten, Abzeichen (Badges), Ranglisten, Levels, Quests oder erzählerischen Elementen wird jedoch ein spielerisches Lernerlebnis geschaffen (PARK UNIVERSITY, 2024). Die Freude und der Antrieb, die Menschen bei Computerspielen empfinden, sollen so für Bildungszwecke nutzbar gemacht werden (PARK UNIVERSITY, 2024). Gamification in der Lehre zielt darauf ab, ähnliche Beteiligungslevels und eine ähnlich tiefe Vertiefung herbeizuführen, wie sie Spiele bei ihren Spielern erreichen (SMIDERLE u. a., 2020). Der Spaß am Spiel wird mit dem Nutzen des Lernens verknüpft. Die Wirksamkeit von Gamification beruht auf mehreren psychologischen Mechanismen. Zunächst nutzt sie Belohnungen und Verstärkung. Durch Punkte und Auszeichnungen erhalten die Lernenden

unmittelbare Anerkennung für ihre Fortschritte (PARK UNIVERSITY, 2024). Diese sofortigen Rückmeldungen aktivieren das Belohnungssystem im Gehirn und verstärken positiv das gewünschte Verhalten, ähnlich wie im Behaviorismus vorhergesagt (SKINNER, 1958). Bereits B.F. Skinner zeigte mit seinen Lernmaschinen (SKINNER, 1958), dass unmittelbares Feedback wesentlich effektiver ist als spätere Rückmeldung, um gewünschte Lernreaktionen zu festigen. Gamifizierte Lernumgebungen knüpfen daran an, indem sie für jede Aktion des Lerners eine unmittelbare Reaktion bereitstellen. Ein zentrales lernförderliches Element der Gamification ist daher das sofortige Feedback. Die Lernenden sehen z.B. direkt, ob eine eingereichte Programmierlösung funktioniert oder ob ein Fehler aufgetreten ist. Diese direkte Rückmeldung hilft, Fehler sofort zu erkennen und zu korrigieren, anstatt falsches Verständnis lange unverändert mitzuschleppen. Bildungsstudien bestätigen, dass solches Zeitlinien-Feedback dem verzögerten überlegen ist und den Lernprozess deutlich verbessert (SINGH, 2022). Gamification-Plattformen bieten durch automatisierte Tests, Live-Auswertungen oder Hinweisfunktionen genau diese Art von Instant-Feedback rund um die Uhr (SINGH, 2022). Darüber hinaus sorgen sie oft für eine dauerhafte Rückkopplung über den Lernfortschritt (z.B. in Form von Fortschrittsbalken, Levels oder Erfahrungspunkten), was den Lernprozess transparenter und selbstbestimmter macht.

Neben dem Feedback spielen Herausforderung und Wettbewerb eine wichtige Rolle. Gut designte Gamification bietet Aufgaben in verschiedenen Schwierigkeitsgraden, oft verpackt als Challenges oder Quests, die den Lernenden fordern, aber nicht überfordern. Diese ansteigende Level-Struktur stellt sicher, dass stets eine Balance zwischen den Fähigkeiten des Lerners und dem Schwierigkeitsgrad der Aufgabe besteht, eine Voraussetzung für das Eintreten des Flow-Zustands(L. LI u. a., 2023). Die Flow-Theorie nach Csíkszentmihályi beschreibt den mentalen Zustand völliger Vertiefung und optimaler Motivation, der auftritt, wenn klare Ziele vorliegen, die Anforderungen gut auf die individuellen Fähigkeiten abgestimmt sind und kontinuierliches Feedback gegeben wird (CHALLCO u. a., 2023). Gamification erfüllt diese Kriterien. Die Lernenden wissen genau, was zu tun ist (durch klare Missionsziele), erhalten ständige Rückmeldungen und erleben durch angemessene Herausforderungen Erfolgserlebnisse. Damit können sie in einen Flow gelangen, in dem Lernen als fast mühelos und hochgradig befriedigend empfunden wird. Dieser hohe Grad an Engagement ist in traditionellen Lernsettings schwer zu erreichen, wird aber durch spielerische Elemente begünstigt (LAMPROPOULOS u. a., 2024).

Gamification spricht sowohl extrinsische als auch intrinsische Motivationsquellen an. Äußere Anreize wie Punkte, Ranglistenplatz oder virtuelle Belohnungen können zunächst die Teilnahmebereitschaft erhöhen. Wichtiger noch, eine gute Gamification-Strategie zielt darauf ab die intrinsische Motivation zu stärken, indem sie grundlegende psychologische Bedürfnisse erfüllt. Hier kommt die Selbstbestimmungstheorie (DETERDING, 2016) ins Spiel. Sie identifiziert Autonomie, Kompetenzerleben und soziale Eingebundenheit als Schlüsselfaktoren intrinsischer Motivation. Gamifizierte Lernumgebungen können alle

drei unterstützen. Die Lernenden agieren oft selbstgesteuert und treffen eigene Entscheidungen (Autonomie), sie meistern schrittweise anspruchsvollere Aufgaben und sehen ihren Kompetenzzuwachs unmittelbar (Kompetenzgefühl durch Feedback), und sie können sich mit anderen messen oder zusammenarbeiten (soziale Eingebundenheit durch Wettbewerbe oder Team-Challenges). Empirische Befunde zeigen, dass Studierende in gamifizierten Kursen diese Bedürfnisse deutlich besser erfüllt sehen, sie fühlen sich kompetenter, autonomer und stärker zu einer Gemeinschaft zugehörig, was zu höherer Zufriedenheit und Motivation führt (LAMPROPOULOS u. a., 2024). So berichten Lampropoulos und Sidiropoulos (2024) in einer Studie, dass die Lernatmosphäre durch Gamification freudvoller und die Grundbedürfnisse der Lernenden nach Kompetenz, Autonomie und Zugehörigkeit erfüllt wurden (LAMPROPOULOS u. a., 2024). Dieses positive Erleben geht mit einer erhöhten Bereitschaft einher, sich mit dem Lernstoff auseinanderzusetzen, auch über das Pflichtmaß hinaus.

Theoretisch lässt sich Gamification mit etablierten Lerntheorien untermauern. Im Behaviorismus wirkt Gamification, wie erwähnt, durch Verstärkung. Korrektes Verhalten (z.B. eine gelöste Aufgabe) wird sofort belohnt, was laut Theorie die Wiederholung solchen Verhaltens fördert. Im Rahmen der kognitiven Lerntheorien lässt sich Gamification als Mittel sehen, das aktive Informationsverarbeitung stimuliert. Durch unmittelbares Feedback und interaktive Aufgaben werden die Lernenden angeregt, ihre Denkstrategien laufend zu reflektieren und anzupassen. Auch Metakognition (das Nachdenken über den eigenen Lösungsweg) kann durch spielerische Selbsttests oder das Freischalten von Hinweisen angeregt werden. Vor allem aber fügt sich Gamification nahtlos in konstruktivistische Ansätze ein. Wissen wird aktiv konstruiert, während der Lernende im Spiel Probleme löst und aus Fehlern lernt, ganz im Sinne von Learning by Doing. Die Gamification-Methode spiegelt somit moderne pädagogische Grundsätze wider, die den Lerner in den Mittelpunkt stellen. Nicht zufällig nennen neuere Übersichtsarbeiten zu E-Learning und Online-Pädagogik Gamification im selben Atemzug mit grundlegenden Lerntheorien wie Behaviorismus, Konstruktivismus, Kognitivismus oder dem Konzept des situativen Lernens (IFTIKHAR u. a., 2022). Gamification ist im Grunde eine didaktische Rahmenstruktur, die diese Theorien praktisch umsetzt. Sie bietet Motivation und Kontext (damit Aufmerksamkeit und Relevanz nach dem ARCS-Modell der Motivation gegeben sind), sorgt für Vertrauen der Lernenden in die eigenen Fähigkeiten durch machbare Schritte (Confidence, ARCS) und stellt Zufriedenheit durch Belohnung und sichtbaren Fortschritt ein. Durch diese breite theoretische Fundierung, von Lerntheorien über Motivationspsychologie bis zu Flow und Verstärkung, kann Gamification als wirkungsvolle Schnittstelle zwischen Spiel und Lernen dienen.

Zusammenfassend bietet Gamification ein didaktisches Werkzeug, um die zuvor beschriebenen Herausforderungen der Programmierausbildung anzugehen. Sie schafft eine Umgebung, in der Anfänger angstfrei experimentieren können (Fehler werden als Lernchancen betrachtet, nicht als Versagen), in der sie ständig üben und sofort Feedback erhalten, und

in der sie motiviert bei der Sache bleiben. Im nächsten Schritt stellt sich die Frage, inwiefern diese Vorteile tatsächlich zu besseren Lernergebnissen führen im Vergleich zu traditionellen Methoden.

2.3 Gamification versus traditionelle Lernmethoden

Angesichts der beschriebenen Merkmale verwundert es nicht, dass Gamification in vielerlei Hinsicht klassische Lehrmethoden übertrifft, besonders in der Programmierausbildung von Anfängern. Traditionelle Lernsettings sind häufig durch Frontalunterricht, Lehrbuchstudium und periodische Prüfungen geprägt. Dabei bleibt die Rolle des Lernenden oft passiv. Man hört zu, liest und bekommt nur zeitverzögert Rückmeldung (z.B. durch benotete Tests oder Aufgaben, oft Tage oder Wochen später). Für Einsteiger in der Programmierung kann diese Konstellation problematisch sein. Die Motivation lässt nach, wenn der Stoff abstrakt bleibt und Erfolgserlebnisse ausbleiben. Tatsächlich wurde der Einsatz herkömmlicher Methoden ohne spielerische oder interaktive Elemente als ein Faktor identifiziert, der Anfänger wenig motiviert und zu geringem Engagement führt (SINGH, 2022). Zudem können in großen Anfangsveranstaltungen individuelle Verständnisschwierigkeiten leicht übersehen werden, das Feedback ist selten personalisiert oder kommt zu spät, um Fehlkonzepte rechtzeitig zu korrigieren.

Gamifizierte Lernansätze adressieren diese Defizite direkt. Motivation und Engagement der Lernenden fallen in Gamification-Szenarien deutlich höher aus als in vergleichbaren traditionellen Kursen. Empirische Studien untermauern dies. So stieg in einer Untersuchung die aktive Beteiligung und Ausdauer der Studenten signifikant, nachdem für einen Programmierkurs ein Gamification-System mit Ranglisten, Punkten und Abzeichen eingeführt worden war. Mehrere Forschungsarbeiten berichten von Zuwächsen bei der Lernzeit, der tieferen Beschäftigung mit dem Stoff und der Kooperation unter den Studierenden, sobald spielerische Elemente integriert wurden (SMIDERLE u. a., 2020). Gamification spricht den Spieltrieb und Wettbewerbsgeist an, was bei vielen Lernenden den „Funken“ zündet, sich intensiver mit dem Lerninhalt auseinanderzusetzen, als dies im traditionellen Rahmen der Fall wäre.

Wichtig ist, dass diese höhere Motivation sich auch in objektiven Lernerfolgen niederschlägt. Eine aktuelle Langzeitstudie von LAMPROPOULOS u. a., 2024 verglich über mehrere Semester einen klassisch unterrichteten Programmierkurs mit einer gamifizierten Variante. Die gamifizierte Gruppe erzielte durchweg bessere Leistungen als die traditionelle Vergleichsgruppe. Konkret lag die Bestehensquote um 14% höher, die durchschnittliche Note um 17% höher, und, vielleicht am bezeichnendsten, die Quote exzellenter Leistungen (hohe Noten) war in der Gamification-Klasse um über 70% höher als im traditionellen Kurs (LAMPROPOULOS u. a., 2024). Auch die Abbruchrate (Drop-out) sank deutlich, was auf eine höhere Lernpersistenz und Zufriedenheit der Studierenden hinweist (LAMPROPOULOS u. a., 2024). Die Autoren folgern, dass Gamification sich als effektiver

Lernansatz erwiesen hat, der gegenüber herkömmlichem Unterricht zu besseren Lernergebnissen, höherer Motivation, gesteigertem Engagement und besseren Abschlussquoten führt (LAMPROPOULOS u. a., 2024). Bemerkenswert ist, dass die Lernenden selbst die gamifizierte Methode als sehr positiv einschätzten und angaben, diese habe ihren Lernprozess bereichert (LAMPROPOULOS u. a., 2024).

Ein zentraler Vorteil von Gamification gegenüber traditioneller Lehre liegt, wie von unserem betreuenden Professor hervorgehoben, in der Kontinuität und Direktheit des Feedbacks. Im klassischen Modell erfahren Studierende häufig erst in der Klausurvorbereitung oder bei der Korrektur einer Hausaufgabe, ob sie den Stoff richtig anwenden können. Bis dahin können sich Fehler unbemerkt verfestigen. Gamifizierte Lernumgebungen hingegen geben sofortige Rückmeldungen bei jedem Schritt, wodurch Lernende unmittelbar aus Fehlern lernen können. Diese Unmittelbarkeit wirkt sich nachweislich positiv auf den Lernerfolg aus, da Verständnisprobleme im Keim erkannt und behoben werden (SINGH, 2022). Zudem erlaubt die kontinuierliche Punkte- oder Fortschrittsanzeige den Studierenden, ihren eigenen Lernfortschritt ständig im Blick zu haben. Das fördert eine reflektierte Lernhaltung. Man weiß genau, welche Inhalte man schon gut beherrscht und wo noch Übung nötig ist, anstatt am Semesterende von einer schlechten Note überrascht zu werden.

Gamification begünstigt auch eine stärker individuell angepasste Lernerfahrung. In einem Spielsystem können Lernende oft in eigenem Tempo voranschreiten, wiederholen, bis eine Aufgabe gemeistert ist oder alternative Wege wählen, etwas, das im Einheitslehrplan nur schwer umzusetzen ist. Für Anfänger mit Grundlagenkenntnissen bedeutet dies, dass sie weder unterfordert noch überfordert werden. Wer schon Vorwissen hat, kann schnellere Fortschritte machen; wer noch Unsicherheiten hat, bekommt durch Wiederholungen und Hilfestellungen die nötige Unterstützung, ohne gleich den Faden zu verlieren. Auf diese Weise wird die Heterogenität der Lernenden besser aufgefangen als in traditionellen Formaten.

Schließlich schafft Gamification eine andere Fehlerkultur. Während im traditionellen Kontext Fehler oft mit Punktabzug und negativer Bewertung verknüpft sind (was bei Studenten Angst und Vermeidungsverhalten auslösen kann), werden Fehler im Spiel-Kontext entdramatisiert. Ein Fehlversuch gehört zum Lernprozess, man kann es sofort noch einmal probieren, fail, learn, retry lautet das Motto. Diese Haltung baut Prüfungsangst ab und fördert resilientes Lernen. Studierende entwickeln die Einstellung, dass Fehlschläge Lerngelegenheiten sind, und bleiben eher am Ball, bis sie eine Lösung gefunden haben. Dadurch verbessern sie nicht nur ihr Fachwissen, sondern auch überfachliche Fähigkeiten wie Durchhaltevermögen und Problemlösekompetenz.

Zusammengefasst bietet Gamification gegenüber traditionellen Methoden deutliche Mehrwerte. Sie motiviert die Lernenden durch spielerische Anreize und direkte Erfolgserlebnisse, sie bietet ständiges Üben in einem attraktiven Kontext, unmittelbares Feedback und personalisierte Lernerfahrungen, und sie fördert eine positive, engagierte Lernat-

mosphäre. All dies ist besonders für Programmieranfänger mit ersten Grundkenntnissen wertvoll, da es genau die Bereiche stärkt, in denen traditionelle Ansätze Schwächen zeigen (Motivation, Anwendungspraxis, Betreuung). Die theoretischen Überlegungen und empirischen Befunde aus diesem Kapitel liefern damit die Grundlage für die Konzeption des geplanten Lernspiels. Ein didaktisches Design, das auf Gamification setzt, verspricht, den Einstieg ins Programmieren effektiver und freudvoller zu gestalten als herkömmliche Lehrmethoden.

3 Stand der Technik: Programmierspiele und Lernplattformen

Im Bereich der Programmierausbildung gibt es eine Vielzahl von Ansätzen, die spielerische und interaktive Elemente einsetzen, um den Lernprozess zu erleichtern. Im folgenden Kapitel wird ein Überblick über den aktuellen Stand der Technik gegeben und zentrale Stärken sowie Schwächen bestehender Lösungen analysiert.

3.1 Blockbasierte Lernumgebungen

Visuelle, blockbasierte Programmierumgebungen wie **Scratch**[3] sind besonders bei jungen Lernenden verbreitet. Scratch verwendet grafische Blöcke, die per Drag-and-Drop miteinander kombiniert werden, wodurch Syntaxfehler vermieden werden und sich Lernende voll auf die logische Struktur ihres Programms konzentrieren können (MONTIEL u. a., 2021). Diese Plattformen sind erwiesenermaßen effektiv, um erstes Verständnis von Algorithmen und Programmstrukturen zu vermitteln und haben eine nachgewiesene positive Wirkung auf die Motivation und das Selbstvertrauen der Lernenden (WEN u. a., 2023). Jedoch stehen sie auch in der Kritik, da Sie die Programmierung zu sehr abstrahieren und dem Lernenden ein falsches Bild vermitteln.
Die blockbasierte Drag-and-Drop-Umgebung von Scratch kann die Vielfalt der Problemlösungsansätze einschränken und damit potenziell sowohl die Kreativität als auch die Entwicklung fortgeschrittener Programmierfähigkeiten limitieren. In dieser gewissermaßen „vakuumkontrollierten Umgebung“ sind die Möglichkeiten zur Erkundung alternativer Lösungswege oder komplexerer logischer Strukturen begrenzt. Einige Forschende argumentieren, dass dies ein tiefergehendes Lernen behindern könne. Gleichwohl wird diese Einschränkung insbesondere auf Primarstufenniveau häufig als akzeptabel betrachtet, da hier die Einführung grundlegender Konzepte gegenüber der inhaltlichen Flexibilität priorisiert wird (MONTIEL u. a., 2021).

[3] Vgl. MIT Media Lab: Scratch. `https://scratch.mit.edu/`, abgerufen am 30.08.2025.

3.2 Puzzle- und Rätselspiele

Ein Spiel wie **Lightbot**[4] präsentiert Programmierkonzepte in Form logischer Rätsel. Spieler lösen Aufgaben, indem sie Befehle an virtuelle Figuren erteilen. Diese Spiele verbessern insbesondere algorithmisches Denken und logische Fähigkeiten (YALLIHEP u. a., 2019).
Stärken:

- Förderung von logischem und algorithmischem Denken.
- Direkte Vermittlung grundlegender Kontrollstrukturen wie Schleifen und Bedingungen.
- Unterhaltsam und motivierend, da spielerisch gestaltet.

Schwächen:

- Oft eingeschränkte Komplexität der Programmieraufgaben.
- Möglicherweise wenig direkte Vorbereitung auf reales Programmieren.

3.3 Code-Adventure-Spiele

Ein prominentes Beispiel ist **CodeCombat**[5], ein Adventure-Spiel, in dem Lernende Programmieraufgaben lösen, um im Spiel voranzukommen. Spieler programmieren in echten Programmiersprachen wie Python oder **JavaScript**[6] und wenden dabei ihre Kenntnisse unmittelbar an (KROUSTALLI u. a., 2021).
CodeCombat kann Lernende motivieren und ihr Interesse am Programmieren steigern. Die Ergebnisse der Studie von (KROUSTALLI u. a., 2021) zeigen, dass der Einsatz von CodeCombat im Informatikunterricht mehrere potenzielle Vorteile aufweist. Ein wesentlicher Aspekt ist die Steigerung der Motivation und des Engagements der Lernenden. In der abschließenden Befragung gaben 75 % der Schülerinnen und Schüler an, dass sie das Lernen von Programmierinhalten mit CodeCombat als gute Idee betrachten, und 65 % berichteten, dass sich ihr Interesse am Programmieren durch den Einsatz des Spiels erhöht habe. Darüber hinaus bewerteten die Teilnehmenden die Benutzerfreundlichkeit und Nützlichkeit der Anwendung überwiegend positiv; 75 % stimmten zu, dass der Unterricht durch CodeCombat unterhaltsamer geworden sei. Die Arbeit mit einer realen textbasierten Programmiersprache wie Python oder JavaScript in einem spielbasierten Umfeld bietet praxisnahes Lernen und erleichtert die Auseinandersetzung mit authentischer Syntax. Durch das unmittelbare Feedback zu Codeeingaben und die integrierten Hilfestellungen im Spiel konnten viele Schülerinnen und Schüler Fehler schnell erkennen und korrigieren;

[4] Vgl. Lightbot: `https://lightbot.com/`, abgerufen am 30.08.2025.
[5] Vgl. CodeCombat: `https://codecombat.com/`, abgerufen am 30.08.2025.
[6] Vgl. Mozilla Developer Network (MDN): `https://developer.mozilla.org/docs/Web/JavaScript`, abgerufen am 30.08.2025.

je nach Item gaben 77,5 % bis 92,5 % der Befragten an, die Tipps und Hinweise als hilfreich empfunden zu haben. Auch frühere Untersuchungen bestätigen, dass CodeCombat Problemlösefähigkeiten fördern kann.
Gleichzeitig verdeutlichen die Studienergebnisse auch Einschränkungen. Obwohl die Experimentalgruppe in den Leistungstests tendenziell bessere Ergebnisse erzielte als die Kontrollgruppe, waren die Unterschiede statistisch nicht signifikant. Zudem besteht die Gefahr, dass sich Lernende stärker auf das Erreichen des nächsten Spiellevels konzentrieren, anstatt die zugrunde liegenden Programmierkonzepte in der nötigen Tiefe zu verstehen. Die Übertragbarkeit der im spielerischen Kontext erworbenen Kenntnisse auf komplexere, nicht-spielerische Programmierumgebungen kann dadurch eingeschränkt sein. Darüber hinaus zeigte sich bei der Nutzungsabsicht ein heterogenes Bild. Nur 35 % der Befragten äußerten den Wunsch, CodeCombat künftig erneut zu nutzen, während 37,5 % dies ablehnten und 27,5 % neutral blieben. Schließlich weist die Studie auch auf praktische Rahmenbedingungen hin, die den Einsatz erschweren können, wie etwa die Abhängigkeit von einer stabilen Internetverbindung sowie mögliche Sprachbarrieren durch die überwiegend englischsprachige Benutzeroberfläche.

3.4 Online-Code-Judge-Plattformen

Plattformen wie **HackerRank**[7], **LeetCode**[8] oder **Codewars**[9] bieten hunderte von Aufgaben, die algorithmische Problemlösungsfähigkeiten fördern. Diese Plattformen nutzen gamifizierte Elemente wie Ranglisten, Punkte und Wettbewerbe, um das Lernen zu motivieren (WASIK, 2018).

Stärken:

- Umfangreiche Aufgabensammlung mit automatischer Bewertung.
- Förderung der Problemlösungsfähigkeit und Programmierfertigkeiten.
- Kompetitive und motivierende Lernumgebung.

Schwächen:

- Starker Fokus auf Wettbewerbsaspekte könnte negative Effekte wie Druck oder Stress erzeugen.
- Weniger geeignet für komplette Anfänger, da komplexes Vorwissen vorausgesetzt wird.

[7] Vgl. HackerRank: `https://www.hackerrank.com/`, abgerufen am 30.08.2025.
[8] Vgl. LeetCode: `https://leetcode.com/`, abgerufen am 30.08.2025.
[9] Vgl. Codewars: `https://www.codewars.com/`, abgerufen am 30.08.2025.

3.5 Fazit: Zusammenfassung und Abgrenzung zum eigenen Ansatz

Die Analyse zeigt, dass existierende Ansätze jeweils spezifische Stärken besitzen, aber auch klare Schwächen aufweisen. Blockbasierte Plattformen sind gut geeignet für Anfänger, vermitteln jedoch selten tiefere Programmierkenntnisse. Rätselspiele fördern logisches Denken, bleiben aber oft zu abstrakt und begrenzt in der Komplexität. Adventure-Spiele verbinden reale Programmiersprachen mit hoher Motivation, können aber schnell überfordernd wirken oder das Lernen in den Hintergrund rücken. Online-Code-Judges sind effektiv für fortgeschrittene Lernende, bieten jedoch wenig Unterstützung für absolute Anfänger.

Der eigene, in dieser Arbeit verfolgte Ansatz grenzt sich klar ab. Es wird ein **Lernspiel** entwickelt, das sich explizit auf die **Festigung bereits vorhandener Kenntnisse** fokussiert. Im Unterschied zu den oben genannten Lösungen steht nicht das Erlernen neuer Syntax oder neuer Konzepte im Vordergrund, sondern die regelmäßige Anwendung und Vertiefung bereits erlernter Inhalte. Ziel ist, die Vorteile existierender Plattformen (z. B. unmittelbares Feedback, gamifizierte Elemente, reale Programmieraufgaben) so zu kombinieren, dass sie genau die identifizierten Lücken bestehender Angebote schließen.

4 Konzeption des Lernspiels

Auf Grundlage der im vorherigen Kapitel ermittelten Anforderungen und der Analyse bestehender Ansätze wird nun das Konzept für ein eigenes Lernspiel vorgestellt. Dieses Spiel richtet sich an Lernende, die bereits grundlegende Programmierkenntnisse besitzen, und soll durch spielerische Herausforderungen dazu beitragen, vorhandenes Wissen effektiv zu festigen und zu vertiefen. Die Ausgestaltung des Spiels folgt bewährten didaktischen Prinzipien und Gamification-Elementen, um Motivation und Lernerfolg zu maximieren.

4.1 Zielgruppe und Lernziele

Die primäre Zielgruppe des Lernspiels umfasst Personen mit grundlegenden bis mittleren Programmierkenntnissen, die bereits Konzepte wie Variablen, Kontrollstrukturen (Bedingungen, Schleifen) sowie einfache Datenstrukturen und Algorithmen beherrschen. Das Spiel ist nicht darauf ausgerichtet, völlig neue Inhalte zu vermitteln, sondern zielt vielmehr darauf ab, vorhandenes Wissen durch praxisorientierte Anwendung und gezielte Übung zu festigen. Dabei stehen drei zentrale Lernziele im Vordergrund. Erstens soll die **Problemlösefähigkeit** gestärkt werden, indem die Spieler durch kreative und logisch aufgebaute Programmieraufgaben gefordert werden, bekannte Konzepte in neuen Kontexten anzuwenden und so ihr Transferdenken zu fördern. Jede Aufgabe stellt ein zu lösendes Problem dar, dessen Bearbeitung die Fähigkeit zum analytischen Problemlösen trainiert. Zweitens wird das **Code-Verständnis** und die Kompetenz im **Debugging** geschult, indem die Lernenden regelmäßig mit fehlerhaftem oder unvollständigem Code konfrontiert werden, den es systematisch zu analysieren und zu korrigieren gilt eine Schlüsselqualifikation für selbstständiges Arbeiten in der Programmierpraxis. Drittens soll das **algorithmische Denken** vertieft werden, indem die Spieler angehalten werden, effiziente Lösungswege zu entwerfen und als Algorithmen umzusetzen, wodurch sie lernen, komplexe Probleme schrittweise in handhabbare Teilprobleme zu zerlegen und diese in korrekten, möglichst effizienten Programmcode zu übertragen. Insgesamt adressieren diese Lernziele wesentliche Kernkompetenzen der Informatik und richten sich an Lernende, die ihre bestehenden Kenntnisse nicht nur anwenden, sondern auch gezielt erweitern und vertiefen möchten.

4.2 Spielmechanik und Designentscheidungen

Um die genannten Lernziele zu erreichen und die Motivation der Lernenden hochzuhalten, kombiniert das Spiel verschiedene spielmechanische Elemente mit didaktischen Methoden. Im Folgenden werden die zentralen Mechaniken und die dahinter stehenden Designentscheidungen vorgestellt und wissenschaftlich begründet.

Unmittelbares Feedback durch Online-Code-Judge

Jede Programmieraufgabe im Spiel wird durch ein automatisiertes Bewertungssystem (Online-Code-Judge) unterstützt. Sobald ein Spieler seinen Code löst und einreicht, führt das System vordefinierte Testfälle aus. Das Ergebnis wird dem Spieler *unmittelbar* zurückgemeldet. Dieses Feedback umfasst Informationen darüber, welche Testfälle erfolgreich waren und wo gegebenenfalls Fehler auftraten (z. B. falsche Ausgabe oder Laufzeitfehler). Durch diese direkte Rückmeldung erkennen die Lernenden sofort, ob ihre Lösung korrekt ist, und können bei Bedarf ihren Ansatz anpassen.
Begründung: Unmittelbares, gezieltes Feedback ist einer der stärksten Einflüsse auf den Lernerfolg (MARWAN u. a., 2022) (HATTIE u. a., 2007). Insbesondere im Programmierkontext ermöglicht sofortiges Feedback ein schnelles Lernen aus Fehlern. Studierende können ihre Lösung iterativ verbessern, ohne lange auf Korrekturen warten zu müssen. Automatisierte Bewertungssysteme in der Programmierausbildung entlasten zudem Lehrende und sorgen für objektive, konsistente Beurteilung. Sie ermöglichen es, vielen Lernenden gleichzeitig zeitnah Rückmeldung zu geben (MARWAN u. a., 2022). Insgesamt fördert die sofortige Fehlerrückmeldung sowohl die Motivation als auch den Lernerfolg, da Erfolge direkt sichtbar werden und Fehlversuche als Lerngelegenheit dienen können.

Levelbasierte Struktur und narrative Rahmung

Das Spiel ist in aufsteigende Levels gegliedert, die mit zunehmender Schwierigkeit komplexere Aufgaben bieten. Die Spieler beginnen mit relativ einfachen Herausforderungen und arbeiten sich schrittweise zu anspruchsvolleren Projekten vor. Diese levelbasierte Progression stellt sicher, dass sie zunächst Erfolgserlebnisse sammeln und dann kontinuierlich gefordert werden, was dem didaktischen Prinzip des „Übergangs vom Leichten zum Schweren“ entspricht.
In das Level-System ist eine narrative Rahmung eingebettet. Die Programmieraufgaben sind in eine fortlaufende Geschichte im Fantasy-Stil eingebunden. Die Spieler schlüpfen beispielsweise in die Rolle eines Zauberlehrlings, der Programmier-Rätsel lösen muss, um ein Königreich vor einer Bedrohung zu retten. Jede abgeschlossene Aufgabe treibt die Geschichte voran und konfrontiert den Spieler mit neuen Szenarien, in denen seine Programmierkenntnisse gefragt sind. Diese erzählerische Einbettung gibt den abstrakten Programmierproblemen einen konkreten Sinnzusammenhang.

Begründung: Eine narrative Gestaltung und klare Level-Struktur sollen die intrinsische Motivation erhöhen und engagiertes Lernen begünstigen. Forschungsarbeiten zum Gamification-Ansatz zeigen, dass Story-Elemente und thematische Rahmenhandlungen als motivationale Spielanreize wirken (ZHAN u. a., 2022). Durch die Geschichte erhält der Spieler Kontext und Ziel für jede Aufgabe, was das Eintauchen in die Lernumgebung erleichtert. Gleichzeitig ermöglichen klar definierte Level eine schrittweise Steigerung der Anforderungen, sodass Überforderung vermieden wird und das Erfolgserlebnis beim Levelaufstieg zu weiterer Teilnahme motiviert (ZHAN u. a., 2022). Insgesamt kombiniert diese Mechanik motivierende Geschichten mit pädagogischem *Scaffolding* (Gerüstbau), um die Spieler kontinuierlich zu fordern, aber nicht zu überfordern.

Belohnungssysteme (Punkte und Badges)

Zur weiteren Steigerung der Motivation integriert das Spiel ein Belohnungssystem. Für gelöste Aufgaben erhalten die Spieler Punkte, die beispielsweise auf einem Punktekonto gutgeschrieben werden. Zusätzlich werden Badges (Abzeichen) als *Achievements* für besondere Leistungen vergeben. Solche Badges kann ein Spieler etwa erhalten, wenn er eine Aufgabe besonders schnell gelöst hat, einen besonders effizienten Algorithmus entwickelt hat oder einen kniffligen versteckten Fehler entdeckt hat. Punkte dienen als unmittelbare Rückmeldung über den Fortschritt, während Badges langfristige Ziele und besondere Erfolgserlebnisse markieren.
Begründung: Der Einsatz von Punkteständen und Abzeichen ist ein etabliertes Gamification-Element in der Bildung. Studien zeigen, dass solche extrinsischen Motivatoren sofern sinnvoll eingesetzt Lernende zum Weitermachen anspornen können (HAMARI u. a., 2014). Insbesondere Badges verleihen bestimmten Leistungen Sichtbarkeit und fördern einen kompetitiven Ehrgeiz, weitere Herausforderungen anzugehen. In einer umfangreichen Literaturübersicht stellten Hamari et al. (2014) fest, dass Gamification-Elemente wie Punkte, Ranglisten und Abzeichen in den meisten Fällen positive Effekte auf Motivation und Engagement der Lernenden haben. Wichtig ist jedoch die didaktische Einbettung. Die Belohnungen im vorliegenden Spiel sind stets an tatsächlich erbrachte Leistungen gekoppelt (z.B. Aufgaben lösen, Fehler beheben), sodass die Lernaktivität selbst im Mittelpunkt bleibt und nicht nur das Sammeln von Punkten. Richtig umgesetzt können solche Belohnungssysteme laut empirischen Untersuchungen das kontinuierliche Üben fördern und die Lernfreude erhöhen.

Wettbewerbsmechanismen (Highscores und Challenges)

Das Lernspiel setzt auf moderaten Wettbewerb als zusätzlichen Anreiz. Einerseits gibt es eine Highscore-Liste, in der Spieler ihre angesammelten Punkte mit anderen vergleichen können. Andererseits werden zeitweilige *Challenges* angeboten beispielsweise Coding-Wettbewerbe, bei denen innerhalb einer bestimmten Zeit eine Aufgabe zu lösen ist. Die

Teilnahme an diesen Wettbewerben ist freiwillig und wird belohnt, jedoch steht der Wettbewerb nicht so im Vordergrund, dass er sozialen Druck erzeugt. Vielmehr soll er als optionaler Motivator dienen für diejenigen, die sich gerne mit anderen messen.
Begründung: Kompetitive Elemente können die Motivation und die Leistungsbereitschaft von Lernenden steigern, wenn sie maßvoll eingesetzt werden. Ein aktuelles Meta-Analyseergebnis aus dem Programmierbildungsbereich zeigt, dass Spiele mit Wettbewerbsmechanismen den größten positiven Einfluss auf die Denkfähigkeiten und die Motivation der Lernenden haben (ZHAN u. a., 2022). Der Reiz des Wettbewerbs spornt viele dazu an, sich intensiver mit dem Problem auseinanderzusetzen und eventuell zusätzliche Zeit ins Üben zu investieren. Gleichzeitig darf der Wettbewerb nicht überwältigend sein, um frustrierte oder ängstliche Reaktionen zu vermeiden. Die Gestaltung als moderate, ergänzende Komponente trägt dem Rechnung. Zhan et al. (2022) fanden, dass insbesondere Ranking-Elemente und kompetitive Spiele in Lernsettings einen signifikanten Motivationsschub bewirken können. Durch die freiwillige Natur der Challenges in dem Spiel bleibt die Lernumgebung jedoch unterstützend und inklusiv niemand wird gezwungen, in Konkurrenz zu treten, aber wer Freude daran hat, bekommt eine zusätzliche Bühne, sein Können zu beweisen.

Sozial- und Ökonomie-System

Spielende können eine *Firma* gründen (allein oder im Team). Jede Firma hat automatisch ein *Bankkonto*. Mit der Firma nimmt man *Aufträge* an (vom System oder von anderen Firmen) oder stellt selbst *Aufgaben* aus. Ein Auftrag ist im Kern eine Programmier-Quest mit kurzer Spezifikation, Abgabe, Tests und klarer Vergütung in *Credits* (plus Erfahrungspunkte für den persönlichen Fortschritt). Firmen verwalten Rollen (z.B. Lead, Entwickler:in, Reviewer:in), Einnahmen/Ausgaben und können mit Credits *Properties* (z. B. Büro/Lab) kaufen, die vor allem als gemeinsamer Raum und zur Personalisierung dienen.
Das *Social*-System macht Zusammenarbeit leicht. Es gibt einen Chat (1:1, Team/Kurs), gemeinsame Aufträge mit Rollen, *Follow* zum Folgen von Freund:innen und *Profile* mit einfachen Privatsphäre-Einstellungen. Auf Profilen stehen verständliche *Stats* wie gelöste Quests, Follower, Firmenzugehörigkeit oder aktuelle Aufträge. Ein *Feed* zeigt wichtige Ereignisse. Abgeschlossene Aufträge, neue Badges, Teammeilensteine oder den Kauf eines neuen Grundstücks/Property mit den verdienten Credits. Beiträge können kurz kommentiert oder geliked werden; der Feed bleibt lernfokussiert (kein Spam).
Die *Ökonomie* bleibt fair und übersichtlich. Erfahrungspunkte sind nicht übertragbar und steuern Level/Freischaltungen; *Credits* sind die weiche Währung für Vergütung, Shop und Properties. Es gibt keine Echtgeld-Käufe und kein Pay-to-Win. Wochen-Caps und sinkende Belohnungen bei Wiederholungen verhindern Grind; Qualität zählt mehr als bloße Menge (Korrektheit, Tests, Effizienz).
Begründung: Die Metaphern „Firma“, „Auftrag“ und „Konto“ holen das Lernen in eine vertraute, reale Struktur und fördern Transfer und Verantwortungsgefühl. Sichtbare, so-

ziale Anerkennung (Profile, Feed) und freiwillige Zusammenarbeit/Wettbewerb erhöhen Motivation und Ausdauer, wenn sie transparent und fair bleiben (HAMARI u. a., 2014; DICHEVA u. a., 2015; ZHAN u. a., 2022).

Spielwelt und Bewegung

Ein zentrales Gestaltungsmerkmal des Lernspiels ist die freie Bewegung der Spielfigur in einer 2D-Pixelwelt, die sich am Top-Down-Stil klassischer Rollenspiele wie **The Legend of Zelda**[10] orientiert. Anstatt Lernaufgaben ausschließlich in Form von Menüs oder abstrakten Interfaces zu präsentieren, werden sie in eine räumlich erfahrbare Spielwelt eingebettet. Spieler:innen steuern ihre Figur mit einfachen Bewegungsbefehlen (z. B. über die Pfeiltasten oder den Analogstick), bewegen sich frei durch die Umgebung und interagieren mit Objekten oder Nicht-Spieler-Charakteren (NPCs).

Die Entscheidung für eine bewegungsbasierte Erkundung verfolgt mehrere didaktische und motivationale Ziele. Erstens fördert sie **Immersion und Motivation**: Durch die Einbettung der Programmieraufgaben in eine begehbare Spielwelt entsteht ein stärkeres Gefühl des „Eintauchens“ in die Lernsituation. Lernende erleben die Aufgaben nicht mehr als isolierte Tests, sondern als Quests, die sie auf ihrer Reise im Spiel entdecken (JARRAH u. a., 2024).

Zweitens unterstützt sie **exploratives Lernen**: Die Möglichkeit, die Spielwelt frei zu erkunden, fördert ein exploratives Lernverhalten. Spieler:innen entscheiden selbst, welchen Bereich sie als nächstes aufsuchen oder welchen Questgeber sie ansprechen. Diese Autonomie steigert die intrinsische Motivation und knüpft an die Selbstbestimmungstheorie von Deci und Ryan an, die Autonomie als wesentlichen Motivationsfaktor beschreibt (DETERDING, 2016).

Drittens stärkt das Design **Nostalgie und Zugänglichkeit**: Der Top-Down-2D-Stil knüpft an bekannte Spieldesigns aus der Kindheit vieler Lernender an. Studien weisen darauf hin, dass nostalgische Bezüge Hemmschwellen senken und positive Emotionen beim Lernen hervorrufen können (MAKAI, 2018). Gleichzeitig ist die reduzierte visuelle Komplexität von Pixelart übersichtlich und verhindert kognitive Überlastung, die Aufmerksamkeit bleibt so auf die Lernaufgaben gerichtet.

Viertens ermöglicht die Bewegung eine **Kontextualisierung von Aufgaben**: Räumliche Bewegung erlaubt es, abstrakte Programmierprobleme narrativ einzubetten. Beispielsweise können fehlerhafte Codes als „verfluchte Schriftrollen“ erscheinen, die im Wald gefunden werden oder eine Firma im Spiel kann neue Aufträge erteilen, die an bestimmten Orten abgeholt werden. Dadurch entsteht ein unmittelbarer Bezug zwischen Lernwelt und Lerninhalt (KRASSMANN u. a., 2017).

Didaktisch bietet die räumliche Verankerung von Quests den Vorteil, dass Lernende Aufgaben in einem sinnstiftenden Kontext wahrnehmen. Die Spielfigur dient als Identifikationsanker, über den der eigene Lernfortschritt in der Welt sichtbar wird. Wer sich erfolgreich

[10] Vgl. Nintendo: The Legend of Zelda. `https://www.zelda.com/`, abgerufen am 30.08.2025.

durch Aufgaben arbeitet, schaltet neue Orte frei, bewegt sich in bislang unzugängliche Bereiche der Karte und verknüpft so räumliche Exploration mit kognitivem Fortschritt. Insgesamt schafft das Bewegungs- und Weltdesign somit eine Lernumgebung, die einerseits vertraut und spielerisch wirkt, andererseits aber gezielt genutzt wird, um Motivation, Autonomie und Transferleistungen zu fördern.

4.3 Wahl der Technologie

Bei der technischen Umsetzung des Lernspiels wurden mehrere Entscheidungen getroffen, um sowohl pädagogische Wirksamkeit als auch eine robuste Software-Architektur sicherzustellen. Im Folgenden werden die wichtigsten Technologie-Entscheidungen vorgestellt und begründet, dazu zählt die Auswahl der Programmiersprache für die Aufgaben, die Wahl der Spiel-Engine zur Entwicklung der Anwendung und die Gestaltung der Code-Ausführungsumgebung für den Online-Judge.

Programmiersprache: Entscheidung für C (vs. Python)

Eine zentrale Gestaltungsfrage war, in welcher Programmiersprache die Lernaufgaben gestellt und gelöst werden sollen. Zur Auswahl standen insbesondere Python und C, zwei Sprachen, die häufig in der Programmierausbildung eingesetzt werden, jedoch sehr unterschiedliche Charakteristika haben. Die Entscheidung fiel darauf, den Schwerpunkt auf C zu legen und Python nur vergleichend zu erwähnen. Diese Wahl wird im Folgenden durch einen Vergleich der Sprachen im Bildungskontext motiviert.
Python wird in vielen Einsteigerkursen bevorzugt, weil die Sprache eine einfache, lesbare Syntax hat und viele komplexe Details (etwa Speicherverwaltung) vor den Nutzern abstrahiert. Dadurch können Anfänger schneller erste Erfolgserlebnisse haben. Studien belegen etwa, dass Einführungskurse in Python tendenziell niedrigere Durchfallquoten und bessere Prüfungsergebnisse aufweisen als solche in C (WAINER u. a., 2018). Python erlaubt es Lernenden, sich auf Problemlösungslogik und Algorithmen zu konzentrieren, ohne vom Syntax-Detailreichtum abgelenkt zu werden. Zudem verfügt Python über einen interaktiven Interpreter und eine Vielzahl von Bibliotheken, was experimentelles Programmieren erleichtert. In pädagogischen Untersuchungen wird Python daher als besonders einsteigerfreundlich und motivationsfördernd beschrieben. Allerdings hat Python auch Nachteile im Kontext dieses Spiels. Durch die hohe Abstraktionsebene der Sprache bleiben viele low-level Konzepte verborgen. Ein Beispiel ist die Speicherverwaltung Python handhabt Speicher automatisch (Garbage Collection), so dass Lernende kaum mit Zeigern oder direkter Speichersteuerung konfrontiert werden. Ebenso sind wichtige Datenstrukturen (Listen, Dictionaries etc.) in Python bereits als High-Level-Konstrukte vorhanden, was zwar anfänglich hilft, aber den Blick auf die Implementierungsdetails versperren kann. Für fortgeschrittene Lernende besteht die Gefahr, dass sie zwar schnell Code schreiben können, aber ein weniger tiefes Verständnis der Vorgänge „unter der Haube“ entwickeln

(BALREIRA u. a., 2022).
C als Programmiersprache stellt demgegenüber höhere Ansprüche an die Lernenden, bietet aber didaktisch auch einzigartige Vorteile. C ist eine *Low-Level*-Sprache, die nahe an der Hardware arbeitet. Lernende müssen hier Konzepte wie Datentypen, Zeiger, manuelle Speicherverwaltung (mit `malloc`/`free`) und effiziente Speicherorganisation verstehen. Diese vermeintliche Hürde kann tatsächlich den Lerneffekt steigern. Wer in C programmiert, erhält ein sehr klares Modell davon, wie ein Computer intern Daten verarbeitet und wie Algorithmen auf Maschinenebene funktionieren. Empirische Untersuchungen deuten beispielsweise darauf hin, dass Studierende durch das Programmieren in C ein leichteres Verständnis für klassische Datenstrukturen entwickeln, weil sie diese in C oft selbst von Grund auf implementieren müssen (BALREIRA u. a., 2022). Während Python viele Strukturen (wie Listen, Queues, Bäume) bereits vordefiniert anbietet, zwingt C dazu, z.B. verkettete Listen oder Arrays manuell zu verwalten ein Prozess, der das algorithmische Denken schult und ein tieferes Verständnis für Speicheraufbau und Datenmanagement vermittelt. Die Kehrseite ist, dass C für Ungeübte als schwieriger gilt. Die Syntax ist komplexer (z.B. erfordert Deklaration von Variablentypen), und Fehler wie Speicherzugriffsfehler oder Pointerfehler können zu Abstürzen führen, was frustrierend sein kann. Tatsächlich zeigen Vergleichsstudien, dass Kurse, die in C unterrichten, im Schnitt höhere Fehlerraten und etwas geringere Anfangserfolge haben als vergleichbare Python-Kurse (WAINER u. a., 2018). Es besteht also ein Spannungsfeld zwischen Tiefe des Verständnisses und Leichtigkeit des Einstiegs.
Für das vorliegende Lernspiel wurde die Entscheidung getroffen, den Fokus auf C zu legen. Die Zielgruppe verfügt bereits über Programmiergrundlagen, so dass die Hürde der C-Syntax überwindbar erscheint. Gleichzeitig verspricht der Einsatz von C einen höheren Lerneffekt im Bereich der System- und Datenstrukturkompetenz, da die Spieler gezwungen sind, sich mit tiefergehenden Konzepten auseinanderzusetzen. Wichtig ist, dass das Spiel durch die beschriebenen Mechanismen (Feedback, Hinweise, etc.) eine unterstützende Lernumgebung bietet, die Frustration abmildert. Python wird im Spiel vergleichend erwähnt und in einzelnen Aufgaben optional unterstützt, um den Transfer zwischen Sprachen zu fördern. Insgesamt jedoch stützt sich das Konzept auf Forschungsergebnisse, die nahelegen, dass fortgeschrittene Programmieranfänger durch die Arbeit mit einer Sprache wie C langfristig profitieren können etwa im Verständnis von Datenstrukturen und in der Vorbereitung auf komplexere Programmierparadigmen (WAINER u. a., 2018). Kurzfristige Nachteile (wie eine anfangs steilere Lernkurve) werden bewusst in Kauf genommen, um mittelfristig einen tieferen Lernerfolg zu erzielen. Diese Entscheidung entspricht auch dem Ziel des Spiels, vorhandenes Wissen zu festigen und zu vertiefen, anstatt lediglich schnelle Erfolgserlebnisse zu produzieren.

Spiel-Engine: Unity

Die Umsetzung der Spielwelt und der interaktiven Elemente erfolgt mit der **Unity-Engine**[11] . Unity ist eine weit verbreitete Game-Engine, die insbesondere für Indie-Spiele und auch für ernsthafte Spiele (Serious Games) im Bildungsbereich häufig genutzt wird. Serious Games sind Spiele, die nicht hauptsächlich der Unterhaltung dienen, sondern einen ernsten Hintergrund haben und darauf abzielen, Wissen, Fähigkeiten oder ein bestimmtes Verhalten zu vermitteln (KANKAANRANTA u. a., 2008). Gründe für die Wahl von Unity sind unter anderem die benutzerfreundliche Entwicklungsumgebung, die große Community und die Plattformunabhängigkeit.

Begründung: Unity eignet sich hervorragend für Lehr- und Lernspiele, da die Engine einen niedrigen Einstieg für Entwickler bietet und zugleich professionelle Möglichkeiten zur Gestaltung komplexer 2D- und 3D-Umgebungen bereithält. Für mein Projekt habe ich mich bewusst für ein 2D-Pixelart-Spiel entschieden. Diese Darstellungsform ist technisch weniger ressourcenintensiv, ermöglicht eine schnelle Prototypenentwicklung und erleichtert den Fokus auf die didaktischen Inhalte anstatt auf aufwendige Grafikproduktion. Gleichzeitig hat Pixelart einen nostalgischen und zugänglichen Charakter, der bei vielen Spielenden positive Assoziationen hervorruft und die Hemmschwelle zum Einstieg senkt (MAKAI, 2018). Durch die klare und reduzierte Darstellung bleibt die Benutzeroberfläche übersichtlich, was insbesondere im Lernkontext wichtig ist, um kognitive Belastung zu reduzieren und die Aufmerksamkeit auf die Lernaufgaben zu lenken (MAKAI, 2018). Zunächst einmal ist Unity für Bildungseinrichtungen kostenlos verfügbar, dadurch werden Kosten eingespart (UNITY TECHNOLOGIES, 2025). Zudem hat es eine geringe Einstiegshürde, plattformübergreifende Unterstützung und den vollen Funktionsumfang einer echten Spiele-Engine. Für das Lernspiel bedeutet dies, dass aufwändige grafische Darstellungen und interaktive Szenarien erstellt werden können, ohne bei null beginnen zu müssen. Bestehende Assets und Bibliotheken (z.B. für Figuren, Physik, UI-Elemente) können genutzt werden, was die Entwicklung effizient macht. Darüber hinaus ist Unity in der Spieleindustrie weit verbreitet, was einen positiven Nebeneffekt darstellt. Lernende, die das Spiel nutzen, kommen so auch mit einer realitätsnahen Technologie in Kontakt, was den Transfer zur Praxis fördern kann. Insgesamt unterstützt Unity als Engine die Zielsetzung, ein motivierendes, ansprechendes Lernspiel bereitzustellen, das technisch stabil auf verschiedensten Endgeräten läuft (SINGH; KAUR, 2022).

Code-Ausführungsumgebung und Online-Judge

Ein zentrales technisches Element des Spiels ist das automatisierte Code-Bewertungssystem (Online Judge), welches die Lösungen der Spieler entgegennimmt und ausführt. Um diese Codeausführung sicher und effizient zu gestalten, wird auf Container-Technologie **Docker**[12] gesetzt. Jede eingereichte Programmierung (insbesondere C-Code) wird serversei-

[11] Vgl. Unity Technologies: `https://unity.com/`, abgerufen am 30.08.2025.
[12] Vgl. Docker Inc.: `https://www.docker.com/`, abgerufen am 30.08.2025.

tig in einem isolierten Docker-Container kompiliert und ausgeführt. Die Container sind so konfiguriert, dass sie nur kontrollierte Ressourcen nutzen dürfen (beschränkte CPU-Zeit, Speicher, kein Netzwerkszugriff etc.), um Missbrauch oder Schäden am Server zu verhindern. Nach Ausführung der Testfälle wird der Container wieder verworfen. Dieses Vorgehen orientiert sich an bewährten Online-Judge-Systemen beispielsweise dem Open-Source-Projekt **Judge0**[13], das ebenfalls containerisierte Sandboxes einsetzt.

Begründung: Die Ausführung von Code birgt immer Sicherheitsrisiken und Herausforderungen in Bezug auf Reproduzierbarkeit und Skalierung. Durch den Einsatz von Docker-Containern wird jeder Programmierlauf in einer abgeschotteten Umgebung durchgeführt, was ein hohes Maß an Sicherheit gewährleistet. Container besitzen ihr eigenes Dateisystem und Prozessraum, isoliert vom Host-System, was verhindert, dass fehlerhafter oder bösartiger Code Schaden anrichtet (COMBE u. a., 2016). Gleichzeitig sind Container skalierbar und sehr ressourceneffizient, da sie sich den Kernel des Host-Betriebssystems teilen und ohne den Overhead vollwertiger virtueller Maschinen auskommen (COMBE u. a., 2016). Das bedeutet, unser System kann bei Bedarf viele Einreichungen parallel verarbeiten, indem einfach mehrere Containerinstanzen gestartet werden. Zudem ist die gesamte Ausführungsumgebung definierbar und zwischen den Spielern identisch, was Chancengleichheit gewährleistet jeder Code läuft unter denselben Bedingungen in einem frischen Container. Unsere Implementierung des Online-Judge mittels Docker knüpft an diese Erkenntnisse an und stellt sicher, dass die didaktisch gewünschte sofortige Rückmeldung technisch zuverlässig und gefahrlos realisiert werden kann.

[13] Vgl. Judge0: `https://judge0.com/`, abgerufen am 30.08.2025.

5 Implementierung des Lernspiels

In diesem Kapitel wird die technische Realisierung des Lernspiels detailliert beschrieben. Zunächst geben wir einen Überblick über die **Systemarchitektur** sowie die Aufteilung in Frontend (Unity-Client) und Backend (Server-seitige Dienste). Anschließend werden die **Backend-Implementierung** und die **Frontend-Implementierung** im Einzelnen erläutert. Schließlich diskutieren wir die umgesetzten Gamification-Elemente, einschließlich des virtuellen Ökonomie-Systems, sowie die sozialen Funktionen und Echtzeit-Features des Spiels. Durch passende Quellverweise wird aufgezeigt, wie das Design unseres Systems im Einklang mit bewährten Konzepten aus der Forschung steht.

5.1 Systemarchitektur

Die Architektur des Lernspiels orientiert sich an einem verteilten Client-Server-Modell, wie es auch in anderen Serious Games häufig eingesetzt wird (KHAN u. a., 2010). Das System ist dabei in mehrere funktional klar getrennte Komponenten unterteilt, die jeweils spezifische Aufgaben übernehmen.

Der **Unity-Client** bildet das eigentliche Frontend des Spiels. Hier erleben die Nutzer die Lernumgebung, steuern ihre Spielfigur, lösen Aufgaben und bedienen die Benutzeroberfläche. Im September 2023 kündigte Unity an, ab 2024 eine neue Preisstruktur einzuführen, bei der Entwickler für jede Installation eines Spiels mit Unity Runtime zahlen sollen (“Unity Runtime Fee"). Die Umsetzung erfolgt mit der Game Engine Unity 2023, die eine effiziente Gestaltung der Spielwelt und der Interaktionselemente ermöglicht und gleichzeitig die neuen Gebühren umgeht. Der Client läuft lokal auf dem Gerät der Spielerinnen und Spieler und kommuniziert über das Internet mit dem Backend-Server, um Daten auszutauschen.

Das Backend ist in **Node.js**[14] mit dem **Express-Framework**[15] realisiert und übernimmt die zentrale Geschäftslogik des Spiels. Mit über einer Million npm-Paketen bietet Node.js Zugriff auf eine Vielzahl an Erweiterungen, Bibliotheken und Tools, von Authentifizierung bis Datenbankanbindung. Express ermöglicht eine schlanke Implementierung von s, Middleware-Logik und Routing-Strukturen, ohne unnötigen Overhead. Es stellt RESTful API-Endpunkte bereit, über die der Client beispielsweise Aufgabenfortschritte abfragen, Code zur Ausführung einsenden oder spielinterne Aktionen wie Käufe und Interaktionen

[14] Vgl. Node.js Foundation: `https://nodejs.org/en`, abgerufen am 27.08.2025.
[15] Vgl. Express.js: `https://expressjs.com/`, abgerufen am 27.08.2025.

anstoßen kann. Darüber hinaus ist der Server für die Authentifizierung der Nutzer und die Autorisierung von Spielaktionen verantwortlich. Die modulare Struktur von Routen und Services unterstützt dabei eine klare Trennung der Funktionalitäten und eine bessere Wartbarkeit.

Für die Datenhaltung und Authentifizierung wird ein **PostgreSQL**[16]-Datenbankserver über **Supabase**[17] als Backend-as-a-Service eingebunden. Supabase bietet neben der Datenbankverwaltung auch eine integrierte Benutzerverwaltung, auf Basis von **JWT-Token**[18] sowie einen Storage-Dienst für Dateien. Spielstände, Kontostände, Quest-Fortschritte oder Bestenlisten werden persistent in der Datenbank gespeichert. Durch den Einsatz von **Row-Level-Security** (RLS)-Regeln ist sichergestellt, dass jeder Spieler ausschließlich auf seine eigenen Datensätze zugreifen kann, was die Datensicherheit erheblich erhöht.

Eine besondere Rolle spielt der **Code-Runner-Dienst**, der für die Ausführung der vom Spieler eingereichten Programmieraufgaben verantwortlich ist. Dieser läuft in separaten **Docker-Containern** und ist über eine **Redis**[19]-basierte Job-Queue mit dem Hauptserver verbunden. Sobald der Client Code zur Überprüfung einschickt, erzeugt der Server einen Job und legt diesen in die Warteschlange. Ein Hintergrundprozess (der Runner) entnimmt die Jobs und führt den Code isoliert im Container aus. Auf diese Weise wird die Codeausführung sicher vom Hauptserver entkoppelt, wodurch sowohl die Skalierbarkeit als auch die Stabilität des Gesamtsystems verbessert werden. Fehlerhafter oder ressourcenintensiver Code kann so die Hauptanwendung nicht beeinträchtigen.

Diese komponentenbasierte Architektur lehnt sich an etablierte Entwürfe für Serious Games und Online-Lernplattformen an. In ähnlichen Forschungsprojekten wird ebenfalls ein mehrschichtiges System verwendet (CHAPLIA u. a., 2023), in dem der Spiele-Client, ein Backend für Benutzerdaten/Auth, sowie ggf. weitere Dienste für spezielle Aufgaben zusammenwirken. Unser Lernspiel benötigt beispielsweise, anders als ein reines Multiplayer-Spiel, keinen dedizierten Echtzeit-Positionsserver (wie einen **Photon-Server**[20]), da die Interaktion zwischen den Spielern eher asynchron über Bestenlisten und Feeds erfolgt. Stattdessen reicht hier eine klassische Zwei-Schichten-Architektur (Client und Server mit Datenbank) aus, ergänzt um den Code-Runner als speziellen Service.

Die **Kommunikation** zwischen Unity-Client und Server erfolgt hauptsächlich über **HTTPS-Requests** gegen die REST-API des Backends. Der Client sendet HTTP-Anfragen (z.B. Login, Abruf der Questliste, Absenden einer Lösung) und erhält **JSON**[21]-Daten als Antwort, die dann im Spiel angezeigt oder weiterverarbeitet werden. Für Echtzeit-Benachrichtigungen, etwa um den Spieler über das Ergebnis eines Code-Runs oder über neue Aktivitäten im

[16] Vgl. PostgreSQL Global Development Group: `https://www.postgresql.org/`, abgerufen am 30.08.2025.

[17] Vgl. Supabase: `https://supabase.com/`, abgerufen am 30.08.2025.

[18] Vgl. JSON Web Token (JWT): `https://jwt.io/`, abgerufen am 30.08.2025.

[19] Vgl. Redis Ltd.: `https://redis.io/`, abgerufen am 30.08.2025.

[20] Vgl. Exit Games: Photon Engine. `https://www.photonengine.com/`, abgerufen am 30.08.2025.

[21] Vgl. ECMA International: ECMA-404 The JSON Data Interchange Standard. `https://www.ecma-international.org/publications-and-standards/standards/ecma-404/`, abgerufen am 30.08.2025.

Spiel zu informieren, nutzt das System **Server-Sent Events** (SSE). SSE ermöglicht es dem Server, fortlaufend Events an den Client zu senden, ohne dass der Client ständig aktiv anfragen muss. Der Unity-Client hält dafür eine dauerhafte Verbindung (HTTP-Stream) zum SSE-Endpunkt aufrecht und reagiert auf empfangene Ereignisse (z.B. „Job abgeschlossen“ oder „Freund X hat eine neue Quest gelöst“). Durch dieses Push-Prinzip bekommt der Spieler nahezu in Echtzeit Rückmeldungen und Spielupdates, was die Benutzererfahrung verbessert. Technisch wird dies umgesetzt, indem der Node.js-Server entsprechende Ereignisse (z.B. den Abschluss eines Code-Jobs oder neue Feed-Events) an alle verbundenen SSE-Clients schickt. WebSockets sind für dieses Spielsystem nicht erforderlich, da Server-Sent Events (SSE) eine effizientere und einfachere Lösung für unidirektionale Echtzeitkommunikation darstellen, bei der lediglich der Server Nachrichten an den Client senden muss, ohne den Overhead und die Komplexität bidirektionaler Verbindungen wie bei WebSockets (TORRE u. a., 2019).
Im Folgenden werden die einzelnen Schichten und Hauptfunktionalitäten, Backend und Frontend, genauer betrachtet.

5.2 Backend-Implementierung

Das Backend des Lernspiels ist als Express-Anwendung in Node.js strukturiert und in verschiedene Module entsprechend den Spielfeatures aufgeteilt. Die wichtigsten Aspekte der Server-Implementierung umfassen die Authentifizierung und Sicherheit, die Verwaltung der Lernszenarien (Quests), die Ausführung eingereichter Codes in isolierten Umgebungen sowie die Ökonomie- und Social-Features des Spiels.

Server-Struktur und Routen

Der Serverstart erfolgt in der Hauptdatei index.js, welche den Express-Webserver aufsetzt, Standard-Middleware lädt und alle Routen registriert. Die Routen sind nach Funktionen gruppiert unter src/routes/ abgelegt, ein gängiges Pattern, um die Codebasis übersichtlich zu halten. So existieren z.B. separate Router für Quests, Challenges, Bank (Banksystem), Shop, Social, Leaderboard usw. Jede dieser Routen-Dateien definiert die HTTP-Endpunkte für das entsprechende Feature. Die API ist zudem per OpenAPI-Spezifikation dokumentiert; ein Swagger UI unter /docs ermöglicht das interaktive Ausprobieren der Endpunkte während der Entwicklung.
Authentifizierung & Autorisierung: Vor den eigentlichen Routen greift eine Auth-Middleware (auth.js), die eintreffende Anfragen abfängt und den mitgesendeten Bearer-Token prüft. Der Token wird vom Unity-Client nach dem Login mitgeschickt (Authorization-Header) und basiert auf dem von Supabase erhaltenen JWT. Die Middleware verifiziert den JWT über den Supabase-Auth-Service und extrahiert die Benutzer-ID sowie Rolleninformationen. So ist im req.user Objekt der authentifizierte Spieler verfügbar. Darauf aufbauend stellt eine weitere Middleware roles.js sicher, dass geschützte Endpunkte nur

von berechtigten Rollen (z.B. „admin“) aufgerufen werden können. Diese Kombination aus JWT-Authentifizierung und Rollenprüfung bildet das Sicherheitsfundament des Backends. Zusätzlich sind in der Datenbank feingranular Row Level Security (RLS)-Regeln definiert, die z.B. bewirken, dass ein Spieler nur Datensätze (Quests, Dateien, Transaktionen etc.) sehen und ändern kann, die ihm selbst gehören. Selbst wenn ein Endpunkt fehlkonfiguriert wäre, verhindert die DB-Ebene somit unberechtigte Zugriffe. Dieses mehrstufige Sicherheitsdesign orientiert sich an Best Practices und schützt sensible Nutzerdaten.
Datenbankintegration: Das Backend verwendet den von Supabase bereitgestellten Postgres-Dienst. Migrationen (Datenbankschemata) und SQL-Views/Functions liegen in src/migrations/ und wurden zu Projektstart angelegt. Wichtige Tabellen sind u.a. users (Spielerprofile), quests (Aufgaben), solutions oder submissions (eingereichte Lösungen), transactions (für das Banksystem), items (Shop-Artikel), properties und companies (spielbare Assets), sowie Tabellen für Freundschaften/Follows und Feed-Nachrichten. Für Echtzeit-Updates kann Supabase Listen auf DB-Änderungen anbieten; in unserem Fall werden jedoch die meisten Echtzeitereignisse entweder über das Node-Backend initiiert (z.B. Feed-Events) oder über Redis-Pub/Sub (siehe Code-Runner unten) abgewickelt. Indizes und Trigger (z.B. ein Trigger `set_updated_at` zur Aktualisierung von Änderungszeitstempeln) sorgen für Performance und Datenkonsistenz.

Umsetzung der Lernszenarien (Quests)

Die Umsetzung der Lernszenarien, im Spiel als Quests bezeichnet, bildet den zentralen Bestandteil des Lernspiels. Jede Quest repräsentiert dabei eine eigenständige Programmier-Challenge, die von den Spielerinnen und Spielern bearbeitet werden muss. Um diese Funktionalität technisch zu realisieren, wurden im Backend spezifische Schnittstellen implementiert, die eine strukturierte Interaktion mit den Aufgaben ermöglichen. Über den Endpunkt GET /quests können die verfügbaren Quests samt ihrer Beschreibungen abgerufen werden. Auf diese Weise erhält der Client die notwendigen Informationen, um die Aufgaben im Spiel darzustellen und die Auswahl durch den Nutzer zu ermöglichen. Sobald ein Spieler eine Aufgabe beginnt, wird dieser Vorgang durch den Endpunkt POST /quests/id/start im System vermerkt. Dadurch wird der Start einer Quest protokolliert und ein individueller Fortschrittsstatus für den Spieler angelegt. Schließlich bietet der Endpunkt POST /quests/id/submit die Möglichkeit, eine Lösung in Form von Programmcode einzureichen. Der eingereichte Code wird anschließend an den Code-Runner übermittelt und dort automatisch anhand vordefinierter Testfälle ausgewertet. Diese Abfolge von Abruf, Start und Abgabe stellt sicher, dass die Bearbeitung der Quests vollständig und konsistent abgebildet wird und gleichzeitig die Grundlage für Feedback, Fortschrittsspeicherung und Gamification-Mechanismen gegeben ist. Jede Quest besitzt in der Datenbank eine Beschreibung des Problems, ggf. Beispiel-Eingaben/-Ausgaben und eine oder mehrere Testfälle, anhand derer die Korrektheit einer Lösung geprüft wird. Wenn ein Spieler Code einreicht, speichert der Server diese Submission und übergibt den Code zur Aus-

führung an den Code-Runner (siehe nächster Abschnitt). Sobald der Runner den Code ausgeführt und das Ergebnis zurückgeliefert hat, bewertet der Server die Ausgabe gegen die erwarteten Lösungen der Testfälle. Ist die Lösung korrekt, wird die Quest für den Spieler als gelöst markiert und Belohnungen (Erfahrungspunkte, virtuelles Geld etc.) werden gutgeschrieben. Bei Fehlschlagen der Tests erhält der Spieler Rückmeldung, welche Tests nicht bestanden wurden, sodass er weiter debuggen kann.
Die automatische Auswertung der Lösungen basiert auf dem Vergleich von Ausgabe oder Rückgabewerten mit den in der Quest hinterlegten Soll-Ergebnissen. Dieses Verfahren entspricht dem Vorgehen gängiger Online-Judge-Systeme und Lernplattformen (HUA u. a., 2020). So können wir objektiv bestimmen, ob ein Spieler die Lernziele der Aufgabe erreicht hat. Der gesamte Prozess, von der Code-Übermittlung bis zur Rückmeldung, läuft für den Nutzer transparent und wird nachfolgend aus technischer Sicht näher erläutert.

Code-Ausführung und -Bewertung (Runner-Service)

Eine Herausforderung bei der Implementierung eines programmierbasierten Lernspiels ist die sichere Ausführung von benutzergeneriertem Code. Spieler könnten absichtlich oder versehentlich Code einsenden, der Endlosschleifen enthält oder die Serverressourcen über Gebühr strapaziert. Um dies zu meistern, wurde ein isoliertes Sandbox-System realisiert. Jedes Programm wird in einem eigenen Docker-Container mit strikten Ressourcenlimits ausgeführt. Dieses Vorgehen ist in der Forschung erprobt. Durch Containerisierung kann verhindert werden, dass schädlicher Code das Host-System beeinträchtigt (ŠPAČEK u. a., 2015). Sobald eine Code-Einsendung erfolgt, weist das System einem vorgehaltenen Container aus einem Docker-Pool die Ausführung zu. Der Code wird darin gestartet; tritt ein Problem auf, wird der Container entfernt, ansonsten steht er nach Abschluss der Ausführung wieder für weitere Einsendungen zur Verfügung. Dadurch bleiben etwaige negative Effekte (z.B. endlose Schleifen oder hohe Auslastung) auf die isolierte Container-Umgebung begrenzt und das System kann sicher weiterbetrieben werden.
Konkret erfolgt die Code-Ausführung im Backend wie folgt.
Job-Erstellung: Bei einer Code-Einsendung erzeugt der Server einen neuen Job mit einer eindeutigen Job-ID. Dieser Job enthält Angaben zur Aufgabe, zur Sprache und natürlich den Code-Text. Der Job wird als JSON in eine Redis-Queue (jobs oder z.B. jobs_c für C-Code) eingestellt. Ausführung in Container: Ein separater Prozess, der Code-Runner, überwacht die Queue. Der Code-Runner ist in Python implementiert (base_runner.py und spielspezifische Unterklassen pro Sprache, z.B. CRunner für C) und nutzt die Docker-Engine. Er hält einen Pool von vorgewärmten Containern bereit, um Wartezeiten zu minimieren, anstatt für jeden Job einen Container neu zu starten (was mehrere Sekunden dauern kann), stehen bereits laufende Containerimages bereit, die nur noch den Code injizieren und ausführen. Der Code-Runner entnimmt also den nächsten Job aus Redis, weist einem vorhandenen Container die Aufgabe zu und führt den Code darin aus. Dabei wird, abhängig von der Sprache, der Code zur Laufzeit kompiliert (z.B. mittels gcc für C)

oder direkt interpretiert. Während der Ausführung gelten restriktive Limits. Der Container hat z.B. nur 256 MB RAM und eine begrenzte CPU-Zeit (Quota, entsprechend max. 5 Sekunden Laufzeit) sowie keinen Netzwerkzugang. Diese Limits sind im Docker-Start des Containers gesetzt (siehe mem_limit, CPU-Quota, etc.) und stellen sicher, dass auch Schleifen nicht den Server lahmlegen. Falls dennoch ein Programm hängen bleibt, kann der Job über einen Cancel-Mechanismus abgebrochen werden, der Server oder Nutzer kann eine Abbruch-Nachricht an den Runner senden, woraufhin dieser den laufenden Containerprozess killt. Anschließend wird der Container verworfen bzw. neu initialisiert, um Seiteneffekte für den nächsten Run auszuschließen.
Ergebnis-Rückgabe: Nach Abschluss (oder Abbruch) der Ausführung schreibt der Runner das Ergebnis (Exit-Code, stdout/stderr Ausgabe, Laufzeit) in Redis zurück unter dem entsprechenden Job-Key. Außerdem wird ein Pub/Sub-Ereignis auf den Kanal job_updates veröffentlicht, das die Job-ID und den Status (done, error etc.) enthält. Der Node.js-Server ist mit diesem Kanal verbunden und erkennt so, dass das Ergebnis vorliegt. Daraufhin kann der Server den Output abrufen und die Überprüfung der Lösung vornehmen. Für den Client wird entweder sofort eine HTTP-Antwort bereitgestellt (falls synchron gewartet wurde) oder, im Regelfall, ein SSE-Event gesendet, um anzuzeigen, dass das Ergebnis verfügbar ist. Durch diese asynchrone Architektur bleibt der Server responsiv, selbst wenn viele Code-Einsendungen parallel laufen, und der Spieler erhält dennoch in kurzer Zeit (<5 Sekunden) sein Ergebnis.
Unser Design unterstützt die Verwendung mehrerer Programmiersprachen. Für jede Sprache kann ein eigenes Docker-Image mit passender Laufzeitumgebung eingesetzt werden (z.B. Python-Interpreter, C-Compiler, usw.). Der Code-Runner verwaltet separate Job-Queues pro Sprache (im Beispiel oben jobs_c für C oder jobs_py für Python) und nutzt jeweils die korrekten Containerimages. So können die Lernenden in verschiedenen Sprachen üben, während der Kern der Ausführungslogik, Warteschlange, Isolation und Rückmeldung, unverändert bleibt. Diese Erweiterbarkeit ist ein wichtiger Aspekt der Implementierung.

Virtuelles Dateisystem und Terminal-Funktion

Um den Spielern ein möglichst realistisches Programmier-Erlebnis zu bieten, stellt das Spiel ein virtuelles Dateisystem mit Terminal-Interface bereit. Hier kann der Nutzer z.B. eigene Skripte oder Dateien abspeichern, Ausgaben anschauen oder einfache Dateikommandos nutzen, ähnlich wie in einer echten Programmierumgebung. Technisch realisiert wird dies durch einen Storage-Service von Supabase (vergleichbar mit S3 von Amazon), der vom Backend über eine Schnittstelle angesprochen wird. Jeder Spieler besitzt einen isolierten Speicherbereich, auf den nur er (und administrativ berechtigte Nutzer) Zugriff haben.
Der Node.js-Server bietet dafür Routen an, die Dateisystem-Operationen kapseln. Z.B. Auflisten von Dateien im „Home“-Verzeichnis (GET /files), Lesen/Schreiben von Datei-

Inhalten (GET/POST /files/path) oder spezielle Terminal-Befehle. Die Terminal-API übersetzt Aufrufe aus dem Unity-Client in entsprechende Storage-Operationen. So wird z.B. ein mkdir-Befehl in einen Storage-Folder anlegen umgesetzt, ein cat file.txt ruft den Dateiinhalt ab, etc. Über das Auth-Token ist sichergestellt, dass jeder nur auf das eigene Verzeichnis zugreifen kann. Damit die Code-Ausführung auch Dateien aus diesem virtuellen Dateisystem nutzen könnte, werden Pfade ggf. auf temporäre URLs oder in den Container gemountete Bereiche abgebildet, wobei in unserer aktuellen Implementierung die Code-Ausführung hauptsächlich den direkt gesendeten Code-String nutzt. Die Terminal-Funktion dient vor allem Lernzwecken (Umgang mit einer Konsole) und zur Persistenz von Lösungsversuchen, die nicht verloren gehen sollen.

Ökonomie-System: Bank, Shop, Grundstücke, Unternehmen

Neben den Coding-Challenges umfasst das Spiel eine Gamification-Ebene mit virtueller Ökonomie. Diese soll die Motivation erhöhen, indem Erfolge in der Programmierung zu Belohnungen in der Spielwelt führen. Im Backend sind hierzu mehrere Komponenten umgesetzt.

Banksystem: Jeder Spieler verfügt über ein virtuelles Bankkonto im Spiel. Auf dieses Konto werden Belohnungen gutgeschrieben, typischerweise verdient der Spieler für das Lösen von Quests oder anderen Achievements eine bestimmte Anzahl an virtuellen Münzen (Coins). Die Konten und Transaktionen werden in der Tabelle transactions bzw. accounts verwaltet. Wenn z.B. eine Quest erfolgreich abgeschlossen wurde, erzeugt das Backend eine Transaktion „Quest-Belohnung“ und erhöht den Kontostand des Spielers um den entsprechenden Betrag. Ebenso können durch Käufe (siehe Shop) oder andere Ausgaben Coins vom Konto abgezogen werden. Durch das Bankmodul lässt sich lückenlos nachvollziehen, welche Aktivitäten zu welchen Kontobewegungen geführt haben (Transaktionshistorie). Das Banksystem ist so gestaltet, dass es erweiterbar ist, weitere Quellen für Einkünfte (z.B. tägliche Login-Boni) oder Ausgaben (z.B. Gebühren) könnten leicht integriert werden.

Shop: Im virtuellen Shop können die erspielten Coins wieder ausgegeben werden. Das Backend hält eine Liste verfügbarer Items bereit (z.B. Upgrades, Gadgets, kosmetische Objekte), die jeweils einen Preis in Coins haben. Über einen API-Endpunkt (POST /shop/buy/item) kann der Client einen Kauf initiieren. Der Server prüft dann, ob der Spieler genügend Guthaben besitzt und das Item verfügbar ist. Ist der Kauf erfolgreich, wird das Item dem Inventar des Spielers hinzugefügt (oder ein Effekt freigeschaltet) und der Kaufpreis als Transaktion vom Konto abgebucht. Auch andere klassische Shop-Mechaniken wie begrenzte Stückzahlen, tägliche Angebote oder Rabatte ließen sich in diesem Rahmen implementieren. Die unmittelbare Rückmeldung beim Kaufen (Erfolg/Misserfolg) erfolgt wieder an den Client, typischerweise als JSON mit Bestätigungsnachricht und aktualisiertem Kontostand.

Grundstücke & Unternehmen: Ein besonderes Feature des Spiels sind Properties (Grund-

stücke) und Companies (Firmen), die Spieler erwerben und verwalten können. Diese bilden eine Art Investitions- und Management-Spiel innerhalb der Lernplattform. Ein Property könnte z.B. ein virtuelles Gebäude sein, das Erträge generiert oder einfach ein Statussymbol. Companies erlauben es Spielern eventuell, sich zu Gruppen zusammenzutun und gemeinsam an etwas zu arbeiten oder Wettbewerb zu betreiben. Implementierungsseitig sind Properties und Companies ebenfalls als Datenbankeinträge realisiert, die gewisse Attribute haben (Eigentümer, Kaufpreis, Ertragsrate, usw.). Spieler können mit genügend virtuellem Kapital solche Objekte kaufen (analog zum Shop-Kauf, aber oft mit höheren Beträgen verbunden). Anschließend könnten sie daraus regelmäßige Gewinne erzielen oder ihren Besitz im Profil anzeigen. Die Unternehmen-Komponente könnte zudem Gruppenaufgaben oder Wettbewerbe ermöglichen, falls mehrere Spieler in einer Firma kooperieren. In der aktuellen Umsetzung liegt der Schwerpunkt jedoch auf dem individuellen Eigentum. Es gibt Endpunkte, um alle verfügbaren Properties/Companies aufzulisten, einen Kauf durchzuführen, und Erträge abzuholen. Damit besitzt das Spiel einen Tycoon-artigen Aspekt, der zusätzlichen Spielspaß bietet und Programmierleistungen belohnt (durch die Währung, die zum Kauf benötigt wird).
Die Einführung solcher Ökonomie-Elemente folgt Gamification-Prinzipien, die in vielen Bildungsanwendungen erfolgreich eingesetzt werden. Punkte, virtuelle Güter und Level gehören zu den am häufigsten genutzten Spielelementen in digitalen Lernsystemen (NICHOLSON, 2013). Indem der Spieler für seine Leistungen unmittelbare Belohnungen in der Spielwelt erhält, entsteht ein zusätzlicher Anreiz, weiterzulernen. Forschungsergebnisse zeigen, dass ein solches Belohnungssystem die Motivation und das Engagement steigern kann (RINCON-FLORES u. a., 2021). Insbesondere wenn die Punkte (hier Coins) und erworbenen Güter sichtbar gemacht werden und in einen Kontext von Wettbewerb oder Sammlung gestellt werden.

Soziale Funktionen und Echtzeit-Feedback

Die Freundes- und Follower-Funktion erlaubt es Spielern, sich im System miteinander zu verbinden. Über Endpunkte wie /social/addFriend/user oder entsprechende GUI-Aktionen im Client können Freundschaften geschlossen oder Nutzer abonniert werden. Die Backend-Logik legt dazu Relationen in einer friends- oder follows-Tabelle an und prüft bei neuen Interaktionen (z.B. Feed-Updates, s.u.), wer diese erhalten darf. Ein Spieler sieht so z.B. bevorzugt die Aktivitäten seiner Freunde.
Eng verknüpft damit ist der Aktivitäten-Feed (Activity Feed oder News-Feed). Immer wenn wichtige Ereignisse im Spiel passieren, z.B. ein Spieler aus der Freundesliste löst eine Quest, erreicht ein neues Level, kauft ein seltenes Item oder gründet eine Company, erzeugt das Backend einen Feed-Eintrag. Dieser enthält eine kurze Nachricht (etwa „Alice hat die Quest 'Arrays sortieren' erfolgreich abgeschlossen"), den Zeitstempel und ggf. Verlinkungen (zur Quest, zu Alice' Profil etc.). Alle verbundenen Clients erhalten dieses Event in Echtzeit per SSE, sofern sie die entsprechenden Personen oder Kategorien abonniert

haben. Die Anzeige dieser Feeds im Spiel fördert ein Wir-Gefühl und eine gegenseitige Anerkennung unter den Lernenden. Die Nutzer können sehen, was andere erreichen, und werden dadurch angespornt, es ihnen gleichzutun oder mit ihnen in Wettbewerb zu treten. Solche sozialen Vergleichs- und Interaktionsmechanismen gelten als wichtiger Erfolgsfaktor von Gamification (SOSNOVSKY u. a., 2020), da sie sowohl kompetitive als auch kollaborative Motivationen ansprechen (Wettbewerb mit anderen vs. gemeinsames Vorankommen).
Ein weiterer wichtiger Bestandteil ist die Bestenliste (Leaderboard). Das Spiel berechnet laufend oder in definierten Intervallen die Rangliste der Spieler anhand bestimmter Kriterien. Typischerweise könnte dies die Summe der verdienten Erfahrungspunkte oder Coins sein, die Anzahl gelöster Quests oder ein kombinierter „Skill-Wert“. Die Leaderboard-API (z.B. GET /leaderboard) liefert die Top-N Spieler und deren Punktestände. Im Backend wird dafür eine effiziente Aggregation verwendet (SQL-Query oder vorberechnete View), um auch bei vielen Spielern die Abfrage schnell zu bedienen. Die Ranglisten werden im Client in einem eigenen Bildschirm angezeigt. Wettbewerbselemente wie Ranglisten sind verbreitete Motivationsinstrumente in Lernspielen, da sie einen Leistungsvergleich ermöglichen und viele Lernende herausfordern, sich zu verbessern (PARK u. a., 2021). Allerdings ist in der Implementierung auch auf Fairness und Datenschutz geachtet. Es werden nur gewünschte Profilinformationen in der Bestenliste gezeigt (z.B. Username, Punkte) und ggf. können Nutzer dem Anzeigen widersprechen, um negativen Druck zu vermeiden. Echtzeit-Feedback: Neben den bereits erwähnten SSE-basierten Feeds und Job-Updates gibt das Backend an vielen Stellen sofortiges Feedback. Beispiele: Nach Absenden einer Lösung erhält der Spieler über die API unmittelbar eine Bestätigung, dass der Code empfangen wurde, und nach kurzer Zeit das Ergebnis der Ausführung. Beim Abschließen einer Quest kann der Server umgehend den neuen Kontostand und eine Erfolgsmeldung zurückspielen. Einkäufe im Shop werden direkt bestätigt und im Inventar sichtbar gemacht. Diese Schnelligkeit wird durch asynchrone Verarbeitung und gezielte Push-Nachrichten erreicht und ist didaktisch wertvoll, da unmittelbares Feedback als förderlich für den Lernprozess gilt.
Zusammenfassend bietet die Backend-Implementierung alle erforderlichen Dienste, um aus der reinen Spielidee ein funktionsfähiges System zu machen. vom sicheren Ausführen und Bewerten von Code über die Verwaltung der virtuellen Welt bis hin zur Integration motivierender sozialer Funktionen. Im nächsten Abschnitt betrachten wir, wie diese serverseitigen Funktionen im Unity-Client präsent und erlebbar gemacht werden.

5.3 Frontend-Implementierung (Unity-Client)

In unserem Projekt dient Unity als Framework für die Benutzeroberfläche, die Spiel-Logik auf Spielerseite sowie die Präsentation aller Gamification-Elemente. Im Folgenden beleuchten wir die wichtigsten Aspekte der Frontend-Implementierung, dazu gehören die

Struktur der Spielszene(n), die UI-Gestaltung für Quests und Coding, die Integration der Netzwerkkommunikation und die Darstellung der Spielmechaniken (Ökonomie, sozialer Feed, Leaderboards) im Client.

Szenen und UI-Struktur

Der Unity-Client ist in mehrere Szenen unterteilt, welche den unterschiedlichen Spielbereichen entsprechen. Nach dem Start der Anwendung landet der Nutzer zunächst im Login/Registrierung-Bildschirm. Hier werden via UI-Formulare die Anmeldedaten erfasst. Unity ruft bei einer Login-Eingabe die Auth-Funktion von Supabase auf (über die Supabase-RESTschnittstelle oder einen entsprechend konfigurierten API-Call an den Backend-Server). Ist die Anmeldung erfolgreich, erhält der Client ein JWT-Token, das für künftige API-Anfragen lokal gespeichert wird (z.B. im Speicher des Geräts) und automatisch jedem Request als Berechtigungsnachweis beigefügt wird.
Nach dem Login gelangt der Spieler in die Hauptspielszene. Diese ist als Hub aufgebaut, von dem aus alle wichtigen Funktionen erreichbar sind. Die Umsetzung kann je nach Design als interaktive Welt oder als Menü-basiertes Dashboard erfolgen. In unserer Implementierung gibt es beispielsweise ein zentrales Spielmenü mit verschiedenen Panels oder Bildschirmen:
Quest-Bereich: Zeigt die Liste der verfügbaren Quests bzw. Lernaufgaben. Jede Quest wird mit Titel, Beschreibung und evtl. Schwierigkeitsgrad aufgelistet. Der Spieler kann eine Quest auswählen, worauf Details eingeblendet werden (z.B. Story-Hintergrund oder Beispiele). Von hier aus gelangt man zur Coding-Umgebung der ausgewählten Aufgabe.
Coding-Interface: Dies ist ein zentraler Bestandteil der Lern-App, hier schreibt der Nutzer den Code zur Lösung der aktuellen Quest. Technisch haben wir in Unity hierfür ein Text-Eingabefeld implementiert, das Mehrzeiligkeit unterstützt (basierend auf Unity's InputField bzw. TMP_InputField für bessere Textdarstellung). Zwar bietet Unity keine eingebaute Code-Editor-Komponente mit Syntax Highlighting, aber durch monospaced Fonts und bestimmte Farbeinstellungen lässt sich zumindest ein editorähnliches Gefühl erreichen. Alternativ könnte ein integriertes Terminal genutzt werden. Unsere Umsetzung erlaubt es, Code einzutippen oder aus dem virtuellen Speicher zu laden. Zusätzlich gibt es Buttons für „Ausführen/Testen“ und „Abschicken“. Klickt der Spieler auf Testen, so wird der aktuelle Code-Text an den Backend-Endpunkt zum Code-Ausführen geschickt (z.B. POST /jobs/run), woraufhin der asynchrone Ablauf (siehe oben) startet. Während der Code vom Server geprüft wird, zeigt die UI einen Ladeindikator. Sobald das SSE-Event mit dem Ergebnis eintrifft, wird entweder die Konsolenausgabe im Interface angezeigt (bei Tests) oder, bei einem Quest-Abschluss, ein entsprechender Erfolg- oder Fehlerschirm. Dieses eingebettete Coding-Interface im Spiel ist ein wesentliches Merkmal des Lernspiels. Vergleichbare Ansätze finden sich etwa im Forschungsprojekt **CodeSpells**[22], wo ein Code-Editor direkt in die 3D-Spielwelt integriert wurde, um Programmieren in einem

[22] Vgl. CodeSpells: `https://codespells.com/`, abgerufen am 30.08.2025.

fantasievollen Kontext erlebbar zu machen.
Terminal-Panel: Ergänzend zum reinen Code-Editor verfügt der Client über ein Terminal-Panel, das eine Eingabezeile und Scrollbereich für Ausgabe hat. Hier kann der Spieler grundlegende Kommandos eingeben, welche an die Terminal-API des Backends geschickt werden. So lassen sich z.B. Dateien im eigenen Spielverzeichnis auflisten (ls), Datei-Inhalte ausgeben oder simple Shell-Befehle simulieren. Das Terminal trägt zur Immersion bei, indem es dem Spieler das Gefühl gibt, innerhalb des Spiels tatsächlich zu „programmieren wie ein Entwickler“. Die Implementation in Unity nutzt ein Textfeld für die Eingabe und ein weiteres Feld für die Ausgabe; Befehle werden beim Abschicken per Web-Anfrage an den Server gesendet, der das Resultat zurückliefert (oder per SSE streamt, falls z.B. ein laufender Prozess Output generiert). Damit fungiert das Terminal als eine Art Mini-Kommandozeile innerhalb des Spiels.
Profil und Bank: In einem weiteren Panel kann der Spieler sein Profil und seinen aktuellen Fortschritt einsehen. Hier wird z.B. der aktuelle Kontostand an Coins angezeigt (der über einen API-Call vom Banksystem geholt wird). Eventuell werden hier auch Statistiken wie Anzahl gelöster Quests, erreichte Levels oder Abzeichen (Badges) dargestellt. Falls das Spiel ein Levelsystem vorsieht (etwa aus Erfahrungspunkten), würde dieses ebenfalls im Profil angezeigt. Die Profilseite bietet dem Nutzer einen Überblick über seine Leistungen und Ressourcen.
Shop-Szene: Öffnet der Spieler den Shop, zeigt die UI eine Liste an Items mit Preis und Beschreibung. Die Items und Preise werden beim Öffnen über die API vom Server geladen (GET /shop/items). Die Spieler können dann per Klick ein Item auswählen und kaufen. Die Umsetzung erfolgt so, dass beim Kauf-Button ein Request an das Backend geht; je nach Antwort (Erfolg/Misserfolg) aktualisiert die UI unmittelbar den neuen Kontostand und ggf. den Besitz des Items. Gekaufte Gegenstände könnten etwa im Profil oder Inventar gelistet werden. Visuell sind die Items mit Icons und Namen repräsentiert, um die Attraktivität der Belohnungen zu unterstreichen.
Eigentum/Firmen: Falls der Spieler Grundstücke oder Unternehmen erworben hat, wird dies in einem eigenen Bereich dargestellt. Möglicherweise gibt es eine Karte oder ein Dashboard, das z.B. gekaufte Gebäude auflistet und ihre Erträge anzeigt. Der Spieler kann hier Erträge einsammeln (was wiederum im Backend einen API-Call auslöst, um Coins gutzuschreiben) oder weitere Grundstücke erwerben. In Unity lässt sich das z.B. mit einer Strategiekarten-Ansicht umsetzen oder simpel als Liste mit „Einsammeln“-Buttons neben jedem Asset. Da diese Features vergleichsweise komplex sind, wurden sie in der ersten Version vor allem als textuelle Darstellung mit Buttons umgesetzt, um die Kernfunktion zu gewährleisten.
Leaderboard und Social Feed: Ein wichtiges UI-Element ist der Live-Feed, der typischerweise am Rand oder als Ticker dargestellt wird. Hier laufen Meldungen über Spieleraktivitäten durch, analog zu sozialen Netzwerken. Unity aktualisiert diesen Feed, sobald über SSE neue Events eintreffen, etwa „Bob hat die Quest XY gelöst (+50 Coins)“. Der Feed ist

scrollbar, sodass man auch vergangene Nachrichten lesen kann. Für die Bestenliste gibt es einen separaten Bildschirm, der auf Abruf die aktuellen Top-Spieler zeigt. Unity erstellt dabei beispielsweise eine scrollbare Rangliste mit Platz, Name und Punktzahl. Durch einen Button kann der Spieler die eigene Position in der Rangliste finden. Die Daten werden via API vom Server bezogen und in UI-Elemente (Textfelder für Name/Punkte) übertragen. Um das Gemeinschaftsgefühl zu stärken, haben wir auch vorgesehen, dass man von der Rangliste oder vom Feed aus direkt auf das Profil eines anderen klicken kann (sofern freigegeben), um z.B. dessen erreichte Badges oder Companies zu sehen.
Benachrichtigungen: Zusätzlich zur Feed-Anzeige nutzt die UI Popup-Meldungen oder Soundeffekte, wenn etwas Wichtiges passiert, zum Beispiel ein Sound und ein kleines Feuerwerk-Animation, wenn der Spieler eine Quest bestanden hat und Coins erhält. Dies gibt positives Feedback und verstärkt die Freude über das Erreichte, was laut Lerntheorie die Motivation erhöhen kann.

Netzwerkkommunikation im Client

Die Integration der Backend-API ins Unity-Frontend erfolgt mittels HTTP- und SSE-Fähigkeiten, die Unity bereitstellt. Für normale API-Requests (etwa JSON-GET/POST) wird UnityWebRequest verwendet. Diese erlaubt asynchrone Webanfragen, die wir in Unity üblicherweise über Coroutines handhaben, d.h. bei einer Anfrage wird gewartet, bis eine Antwort eintrifft, dann wird die Coroutine fortgesetzt und die Antwort verarbeitet. Die vom Server kommenden JSON-Daten werden mit Unitys JSON-Utility oder einem Parser (z.B. Newtonsoft JSON) in **C#**[23]-Datenklassen umgewandelt. So kann der Client z.B. die erhaltene Questliste in eine Liste von Quest-Objekten (mit Feldern für Beschreibung, Belohnung etc.) umwandeln und diese dann rendern.
Die SSE-Verbindung wird in Unity über einen dauerhaften UnityWebRequest-Stream realisiert. Unity ermöglicht das Lesen von Datenstreams in einer Coroutine, wodurch eingehende Events schrittweise verarbeitet werden können. Eine andere Option ist die Nutzung von **.NET**[24]-Standardklassen (z.B. HttpClient mit HttpCompletionOption.StreamResponse) innerhalb von C#, was aber in Unity-Umgebungen mit Vorsicht zu behandeln ist. Wir haben uns für eine erprobte Unity-Lösung entschieden. Ein Hintergrund-GameObject öffnet beim Login eine SSE-Verbindung zum Server (z.B. Endpoint /feed/stream) und lauscht permanent. Jedes Mal, wenn ein Datenblock empfangen wird (typischerweise in SSE im Format event: X: ...), wird dieser ausgewertet und je nach Event-Typ an die zuständigen UI-Komponenten weitergereicht. Beispielsweise sorgt ein Event job_done mit einer Quest-ID und Ergebnis dafür, dass das Quest-Interface das Resultat anzeigt; ein Event new_feed mit einer Nachricht fügt dem Feed-Panel einen neuen Eintrag hinzu. Durch diese Architektur reagiert der Unity-Client ereignisgetrieben und kann viele Vorgänge parallel behandeln,

[23] Vgl. ECMA International: ECMA-334 - C# Language Specification. `https://www.ecma-international.org/publications-and-standards/standards/ecma-334/`, abgerufen am 30.08.2025.
[24] Vgl. Microsoft: .NET Plattform. `https://dotnet.microsoft.com/`, abgerufen am 30.08.2025.

ohne ständig aktiv nachfragen zu müssen.
In Bezug auf die Performance ist zu erwähnen, dass Unity-Apps auf kontinuierliche 3D/2D-Rendering-Schleifen ausgerichtet sind, was bedeutet, dass UI-Updates und Netzwerkcallbacks im Spiel-Loop (jedes Frame) behandelt werden können. Die hier beschriebenen UI-Aktualisierungen (Quest abgeschlossen, Feed-Update etc.) sind seltene Ereignisse im Vergleich zur Frame-Rate und beeinflussen die Performance kaum. Wichtiger war es, die Nebenläufigkeit korrekt zu handhaben, z.B. darf die SSE-Coroutine nicht den Hauptthread blockieren. Dies wurde durch geeignete Programmierung (asynchrone Verarbeitung, Nutzung von Unity Main Thread für UI-Änderungen via UnityMainThreadDispatcher falls nötig) sichergestellt.

Darstellung der Gamification-Elemente

Alle zuvor im Backend beschriebenen Gamification-Mechaniken müssen im Frontend sichtbar und erlebbar sein, um ihre volle Wirkung zu entfalten. Unity bietet reichhaltige Möglichkeiten, diese Elemente attraktiv umzusetzen:
Punktestände und Währung: Der aktuelle Punktestand (Coins) des Spielers wird ständig in der HUD oder im Hauptmenü angezeigt. Ein Münz-Icon mit Zahl rechtsoben im Bildschirm informiert den Spieler jederzeit über sein Guthaben. Steigt das Guthaben (z.B. durch Quest-Erfolg), lässt eine kurze Animation die Zahl ansteigen oder hebt sie farblich hervor. Damit werden Belohnungen sofort und positiv vermittelt, was motivationsfördernd ist (RINCON-FLORES u. a., 2021).
Fortschrittsanzeige: Falls der Spieler Erfahrungspunkte (EP) sammelt und Level aufsteigt, zeigt ein EP-Balken den Fortschritt zum nächsten Level. Bei Abschluss einer Aufgabe füllt sich der Balken anteilig und es erscheint ggf. die Meldung „Levelaufstieg!“ mit Konfetti-Effekt, sobald ein neues Level erreicht wurde. Solche Level und Progress Bars sind in Gamification oft genutzt (MAZARAKIS u. a., 2020), weil sie langfristige Ziele setzen und den Lernfortschritt visualisieren.
Abzeichen (Badges): Für besondere Leistungen (z.B. 5 Quests in Folge korrekt gelöst, erster Code in C geschrieben, 100 Kompilierungen) können Badges vergeben werden. In der UI tauchen diese als kleine Abzeichen im Profil des Spielers auf. Neue Badges werden unmittelbar nach Erreichen in einer Benachrichtigung angekündigt („Badge erhalten: Fleißiger Programmierer!“) und im Profil markiert. Auch diese dienen der Anerkennung und steigern nachweislich die Motivation in E-Learning-Settings („Gamifying Education: Motivation and the Implementation of Digital Badges for Use in Higher Education“, 2018).
Shop und Items: Gekaufte Items werden grafisch dargestellt, z.B. als Icons im Inventar oder als besondere Fähigkeiten. Ein kosmetisches Item könnte z.B. das Aussehen des Avatars verändern, Unity erleichtert dies durch dynamisches Laden von Sprites oder 3D-Modellen. Funktionale Items (z.B. „Verdopple deine nächsten Quest-Coins“) werden im Hintergrund vom Gameplay-Manager berücksichtigt. Wichtig ist, dass der Spieler den Wert seiner ge-

kauften Gegenstände sieht und nutzen kann, da dies den Belohnungskreislauf schließt.
Eigentum & Firmen: Wenn vorhanden, werden Grundstücke möglicherweise auf einer Spielkarte angezeigt (z.B. kleine Gebäude-Symbole auf einer Map) mit dem Namen des Eigentümers. Unitys UI-System kann beispielsweise kleine Tooltip-Fenster anzeigen, wenn man ein Gebäude hovert, und Informationen wie „Gehört Alice, generiert 5 Coins/Std“ einblenden. Für Unternehmen könnte es Ranglisten der erfolgreichsten Unternehmen geben oder spezielle Team-Chats. Der aktuelle Prototyp beschränkt sich jedoch auf die textuelle Darstellung der eigenen Grundstücke im Profil.
Sozialer Feed und Rangliste: Der Feed ist ständig im Blickfeld des Nutzers (z.B. als seitlich einblendbare Leiste). Jeder Feed-Eintrag wird mit kleinen Porträtbildern oder Icons versehen, um die Art der Nachricht erkennbar zu machen (für Erfolge, für Quest-Abschluss etc.). Die Rangliste ist über einen Button erreichbar; dort kann der Spieler scrollen. Die Top 3 werden vielleicht besonders hervorgehoben (z.B. mit Gold-, Silber-, Bronze-Farben), was den kompetitiven Anreiz erhöht. Gleichzeitig achten wir auf eine ausgewogene Darstellung, um Neulinge nicht abzuschrecken, z.B. könnte das Spiel anstelle absoluten Rankings auch persönliche Ziele anzeigen („Du bist besser als 70% der Spieler“), um kontinuierliche Motivation für alle Leistungsniveaus zu gewährleisten (PARK u. a., 2021).

Spielfigur und Movement-System

Die Umsetzung der freien Bewegung der Spielfigur in der 2D-Pixelwelt erfolgte mit Hilfe der Unity-Engine und dem integrierten 2D-Physics- und Input-System. Die Spielfigur wird als *GameObject* mit einem *Rigidbody2D* und einem *Collider2D* modelliert, sodass Kollisionen mit der Spielumgebung (z. B. Wände, Bäume, Objekte) zuverlässig erkannt und realistisch behandelt werden. Die Steuerung erfolgt über das neue Unity Input System, wodurch sich Eingaben sowohl von Tastatur als auch von Gamepads nahtlos verarbeiten lassen.
Die Spielfigur reagiert auf horizontale und vertikale Eingaben, die in jedem Frame als Bewegungsvektor erfasst werden. Dieser Vektor wird normalisiert und anschließend auf das *Rigidbody2D* angewendet, sodass die Figur sich gleichmäßig in alle Richtungen bewegen kann. Das System unterstützt sowohl reine Richtungsbewegungen (oben, unten, links, rechts) als auch diagonale Bewegungen, wodurch ein flüssiges Top-Down-Bewegungsgefühl entsteht, das klassischen *Zelda*-Spielen nachempfunden ist.
Die Spielwelt selbst ist als Tilemap aufgebaut, die in Unity segmentiert geladen wird. Dadurch lassen sich unterschiedliche Kartenbereiche (z. B. Stadt, Wald, Dungeon) modular erstellen und kombinieren. Kollisionseigenschaften können pro Tile definiert werden, sodass bestimmte Bereiche für die Spielfigur unzugänglich bleiben, bis sie durch das Lösen von Quests oder durch Spielfortschritt freigeschaltet werden.
Zur Steigerung der Immersion wurde ein Animation Controller implementiert, der Bewegungen der Spielfigur mit Laufanimationen in vier Richtungen (oben, unten, links, rechts) darstellt. Befindet sich die Spielfigur im Stillstand, wird automatisch die Idle-Animation

ausgelöst. Eine kleine technische Herausforderung bestand darin, ein „Flackern“ zwischen Idle- und Laufanimationen zu verhindern, wenn Eingaben nur sehr kurz oder ungenau erfolgten. Dieses Problem wurde durch die Einführung einer Deadzone für Input-Vektoren sowie einer kurzen Pufferzeit (*Chord Window*) gelöst, bevor zwischen Animationen gewechselt wird.
Die Bewegung ist direkt mit dem Quest- und Interaktionssystem verknüpft. NPCs oder Objekte in der Welt besitzen eigene Collider und ein Skript für Interaktionen. Nähert sich die Spielfigur einem solchen Objekt, kann die Interaktionstaste gedrückt werden, um eine Quest zu starten oder Informationen zu erhalten. Damit fungiert die Bewegung nicht nur als Navigationsmechanik, sondern auch als zentrale Schnittstelle zwischen Lernenden und den didaktischen Inhalten des Spiels.

Technische Herausforderungen und Lösungen

Bei der Frontend-Implementierung mussten einige technische Hürden gemeistert werden. Eine davon war die Integration der Programmierfunktion in Unity. Da Unity primär für Spielelogik und Grafik ausgelegt ist, war das Einbetten eines Texteditors/Terminals nicht trivial. Dies wurde mit vorhandenen UI-Komponenten und etwas Kreativität gelöst, wie oben beschrieben. Ein weiterer Punkt war die Synchronisation mit dem Backend. Die Latenz der Codeausführung und Netzwerkkommunikation erforderte, dem Spieler die Wartezeit angenehm zu gestalten (Lade-Animationen, ggf. Zwischenergebnisse anzeigen). Zudem musste die Unity-App robust gegen Netzwerkfehler sein, z.B. wird beim Verlust der Verbindung zum Server der Nutzer informiert und es wird versucht, die Verbindung neu aufzubauen, ohne dass Spielfortschritte verloren gehen.
Auch die Testbarkeit des Frontends war ein Aspekt. Für Kapitel 6 (technische Validierung) ist wichtig, dass wir bestimmte Aktionen automatisiert prüfen können. Daher haben wir einige Debug-Schnittstellen eingebaut (z.B. Tasten, die direkt einen bestimmten Quest-Zustand laden), um Testfälle schnell anzusteuern. Unitys Event-System erleichterte es außerdem, die UI-Komponenten zu entkoppeln, sodass Unit-Tests auf einzelne Logikklassen ,etwa die Klasse die SSE-Events verarbeitet, geschrieben werden konnten.
Abschließend ist festzuhalten, dass die Frontend-Implementierung alle erforderlichen Elemente enthält, um das Lernspiel benutzerfreundlich, interaktiv und motivierend zu gestalten. Durch Unity konnten wir ansprechende Visualisierungen umsetzen, während die Backend-Anbindung für inhaltliche Substanz sorgt. Die Kombination aus Programmieren im Spiel, sozialer Interaktion und Belohnungssystem ist sorgfältig austariert, um ein motivierendes Lernumfeld zu schaffen. Im nächsten Kapitel (Kapitel 6) wird geprüft, inwieweit diese Implementierung technisch korrekt funktioniert und die gesteckten Anforderungen erfüllt, anhand definierter Testfälle und Kriterien der technischen Validierung.

6 Technische Validierung des Lernspiels

In diesem Kapitel wird die technische Validierung des entwickelten E-Learning-Spiels beschrieben. Zunächst werden die Testfälle und der Online-Code-Judge erläutert, mit denen die korrekte Funktionalität der Programmieraufgaben sichergestellt wurde. Anschließend wird eine empirische Studie skizziert, um zu prüfen, ob der Gamification-Ansatz tatsächlich zu einer höheren Lernproduktivität in der Programmierausbildung führt.

6.1 Testfälle zur Validierung des Online-Code-Judges

Ein zentrales Element der technischen Validierung ist der Einsatz eines Online-Code-Judge zur automatischen Bewertung der Lösungen. Jeder vom Lernspiel gestellte Programmierauftrag wurde mit einer Sammlung von Testfällen verifiziert, um die Korrektheit und Robustheit der Lösungen sicherzustellen. In der Literatur wird betont, dass Testfall-Sammlungen auch Randfälle und unübliche Eingaben umfassen sollten, um die Studierenden zu "bullet-proof"-Lösungen anzuhalten (Z. LI u. a., 2020). Entsprechend wurden für jede Aufgabe neben regulären Beispielrechnungen auch Grenzwerte und Sonderfälle als Testeingaben definiert. Studien zeigen, dass in automatischen Bewertungssystemen pro Programmieraufgabe häufig mindestens 10 Testfälle (inklusive versteckter secret Tests) eingesetzt werden, um alle Aspekte der Lösung umfassend zu prüfen (Z. LI u. a., 2020).
Der Online-Judge führt die eingereichten Programme gegen alle hinterlegten Testfälle aus und vergleicht die Programm-Ausgaben mit den erwarteten Ergebnissen. Jeder Testlauf wird dabei automatisch klassifiziert, z.B. als "accepted" (Test bestanden), "wrong answer" (falsche Antwort) oder "time limit exceeded" (Zeitlimit überschritten). Auf diese Weise erhalten die Lernenden unmittelbares Feedback zu jeder eingereichten Lösung und wissen genau, welche Testfälle ggf. fehlgeschlagen sind. Dies fördert iterative Verbesserungen und stellt sicher, dass die Lösungen nicht nur für einzelne Beispiele, sondern für alle definierten Eingabefälle korrekt funktionieren (LATIH u. a., 2017). Angenommen, eine Übungsaufgabe verlangt das Einlesen zweier Zahlen und das Ausgeben ihrer Summe. Mögliche Testfälle für diese Aufgabe wären:

- Testfall 1 (Normalfall): Eingabe: 3 und 5 (jeweils in separaten Zeilen); erwartete Ausgabe: 8.

- Testfall 2 (Randfall mit Nullwerten): Eingabe: 0 und 0; erwartete Ausgabe: 0.
- Testfall 3 (Negativzahl über STDIN): Eingabe: -4 und 10; erwartete Ausgabe: 6.

Diese Testfälle prüfen sowohl einen typischen Fall als auch Sonderfälle wie Null- und Negativwerte. Insbesondere Testfall 3 demonstriert die Behandlung von STDIN-Eingaben über mehrere Zeilen, der Online-Judge stellt dem Programm hier eine sequenzielle Eingabe bereit (erste Zeile -4, zweite Zeile 10), die das Programm korrekt verarbeiten muss. Alle Aufgaben des Lernspiels wurden analog mit vielfältigen Eingabemöglichkeiten getestet, um sicherzustellen, dass die Studierenden-Einreichungen in jedem Szenario das richtige Ergebnis liefern. Neben der reinen Ergebnisvalidierung der Programmierlösungen wurde auch das korrekte Funktionieren der Gamification-Elemente überprüft. Das Lernspiel umfasst klassische Gamification-Mechaniken analog zu etablierten Plattformen, nämlich Punkte, Quests/Level, Abzeichen (Badges), Fortschrittsbalken und Bestenlisten (FLORES u. a., 2023). In der technischen Validierung wurde getestet, ob diese Mechaniken wie vorgesehen greifen. Beispielsweise musste nach Abschluss einer Aufgabe die entsprechende Punktzahl auf dem Nutzerkonto gutgeschrieben werden und ggf. ein Badge für das Erreichen eines Meilensteins freigeschaltet werden. Auch die Aktualisierung der Bestenliste nach neuen Punkten wurde verifiziert. Durch manuelle Tests solcher Anwendungsfälle konnte sichergestellt werden, dass alle Komponenten (Aufgabenbewertung, Punktezuteilung, Statusanzeigen) konsistent zusammenwirken und der Spielzustand jederzeit korrekt ist. Insgesamt garantieren diese Maßnahmen, dass das Lernspiel technisch zuverlässig funktioniert und Fehler in der Aufgabenprüfung oder Gamification-Logik ausgeschlossen werden können.

6.2 Validierung des Movement-Systems

Zur technischen Validierung des Movement-Systems wurden gezielte Testfälle entworfen, um die Stabilität und Funktionsfähigkeit der Zelda-artigen Spielfigur-Bewegung sicherzustellen. Dabei wurden verschiedene Aspekte berücksichtigt.
Zunächst stand die **Bewegung und Kollisionen** im Fokus. Die Spielfigur wurde in allen vier Hauptrichtungen sowie diagonal bewegt, um die Gleichmäßigkeit der Bewegung zu prüfen. Besonders wichtig war hierbei die Überprüfung, ob Bewegungen an Tilemap-Grenzen zuverlässig durch Collider blockiert werden. In unterschiedlichen Szenen, wie etwa Stadtumgebungen oder Dungeons, ließ sich bestätigen, dass die Kollisionsabfrage konsistent funktioniert und keine unbeabsichtigten Durchgänge oder „Clipping-Effekte“ auftreten.
Ein weiterer Schwerpunkt lag auf den **Animationen**. Ein spezieller Testfall validierte die Übergänge zwischen Idle- und Laufanimationen. Dank einer eingebauten Eingabe-Toleranz (Deadzone) und einer kurzen Verzögerung beim Animationswechsel (Puffer) wird verhindert, dass die Figur bei minimalen oder sehr kurzen Eingaben ständig zwischen

„Stehen“ und „Laufen“ hin- und herspringt. Dadurch laufen die Bewegungsanimationen flüssig und ohne störendes Flackern. Spieler:innen erleben dadurch eine stabile Darstellung, die die Immersion unterstützt (Z. LI u. a., 2020).
Abschließend wurde die **Quest-Interaktion** geprüft. Dabei galt es zu überprüfen, ob Quests zuverlässig durch Interaktionen ausgelöst werden, sobald die Spielfigur ein NPC-Objekt erreicht. Alle Testläufe bestätigten, dass die räumliche Bewegung korrekt als Auslöser für die didaktischen Aufgaben dient.
Die Validierungsergebnisse zeigen insgesamt, dass das Movement-System nicht nur technisch stabil, sondern auch funktional zuverlässig ist. Damit erfüllt es die zentrale Rolle, die begehbare Spielwelt als Interface für die Lernaufgaben verfügbar zu machen.

6.3 Geplante Studie: Gamifizierter vs. klassischer Lernansatz

Obwohl das System technisch einwandfrei arbeitet, stellt sich die Frage, ob der spielerische Gamification-Ansatz tatsächlich einen pädagogischen Mehrwert bietet. Neu eingeführte gamifizierte Lernplattformen sollten unbedingt auf ihre Lerneffekte hin evaluiert werden. Daher soll die folgende Hypothese überprüft werden: “Gamification in Form eines spielerischen Ansatzes (E-Learning Game) erhöht die Lernproduktivität in der Programmierausbildung signifikant.” Um diese Hypothese zu validieren, wird eine vergleichende empirische Studie zwischen dem E-Learning-Game und einer traditionellen Lehrmethode konzipiert. Das Studiendesign kann wie folgt skizziert werden.
Um die Hypothese zu überprüfen, dass ein spielerischer Ansatz die Lernproduktivität in der Programmierausbildung signifikant steigert, wird eine vergleichende empirische Studie zwischen dem E-Learning-Game und einer traditionellen Lehrmethode konzipiert. Geplant ist eine Stichprobe von etwa 50 bis 60 Bachelor-Studierenden der Informatik im dritten Semester, die bereits über grundlegende Programmierkenntnisse verfügen, wie sie in vorherigen Einführungsveranstaltungen erworben wurden. Diese Voraussetzung stellt sicher, dass alle Teilnehmenden in der Lage sind, die im Experiment gestellten Aufgaben zu bearbeiten. Die Studierenden werden nach dem Zufallsprinzip in zwei gleich große Gruppen aufgeteilt. Die Experimentalgruppe nutzt über einen definierten Zeitraum das entwickelte E-Learning-Game als Lernwerkzeug, während die Kontrollgruppe die gleichen Inhalte auf klassische Weise, etwa mit Übungsblättern und Musterlösungen oder in einem Online-Praktikum ohne Gamification-Elemente, erarbeitet. Durch die Randomisierung wird gewährleistet, dass beide Gruppen hinsichtlich Leistungsniveau, Motivation und anderen relevanten Faktoren im Durchschnitt vergleichbar sind, sodass Unterschiede in den Ergebnissen auf die jeweilige Lehrmethode zurückgeführt werden können.
Die Lernphase erstreckt sich über mehrere Wochen, typischerweise vier bis sechs, und umfasst identische Lehrinhalte und Aufgabenstellungen für beide Gruppen. Der entscheidende Unterschied liegt ausschließlich in der Vermittlungsform. Während die Experimentalgrup-

pe im spielerischen Umfeld des E-Learning-Games mit unmittelbarem Feedback, Punktesystemen und Wettbewerbsaspekten arbeitet, bearbeitet die Kontrollgruppe die gleichen Aufgaben in einer traditionellen, nicht-gamifizierten Umgebung. Beide Gruppen erhalten gleich viel Zeit und denselben fachlichen Input, wodurch die Rahmenbedingungen vergleichbar bleiben und lediglich der Einfluss der Gamification isoliert untersucht werden kann.

Die Messung der Lernproduktivität erfolgt durch die Kombination mehrerer Ansätze. Zunächst absolvieren alle Teilnehmenden einen Vortest, der den Ausgangsstand ihrer Programmierkenntnisse erfasst. Nach Abschluss der Lernphase folgt ein Nachtest mit inhaltlich vergleichbaren Aufgaben, um den Wissenszuwachs zu bestimmen. Die Differenz zwischen Vor- und Nachtest dient als primäre Erfolgsgröße. Darüber hinaus wird erhoben, wie viele Übungsaufgaben die Studierenden während der Lernphase erfolgreich gelöst haben, da ein höheres Aufgabenvolumen als Indikator für effizienteres Lernen gewertet werden kann. Um ein umfassendes Bild zu erhalten, werden sowohl die kontinuierlichen Leistungen während der Lernphase als auch die Ergebnisse der Abschlussprüfung betrachtet.

Die Auswertung erfolgt durch den statistischen Vergleich der beiden Gruppen. Dabei wird insbesondere die Leistungssteigerung zwischen Vor- und Nachtest sowie die Anzahl gelöster Aufgaben gegenübergestellt. Zur Analyse bietet sich ein t-Test für unabhängige Stichproben an, um festzustellen, ob die Unterschiede zwischen Experimental- und Kontrollgruppe signifikant sind. Zusätzlich sollen Effektstärken berechnet werden, um die Größe des Gamification-Effekts zu quantifizieren. Ergänzend ist vorgesehen, die Teilnehmenden nach Abschluss qualitativ zu befragen, um Einblicke in ihre Motivation und Zufriedenheit mit der Lernmethode zu erhalten. Diese qualitativen Rückmeldungen ergänzen die quantitativen Messungen, auch wenn sie nicht unmittelbar zur Kernmetrik der Lernproduktivität gehören. Insgesamt erlaubt dieses Studiendesign eine fundierte Überprüfung der Hypothese und liefert belastbare Daten über den Mehrwert gamifizierter Ansätze in der Programmierausbildung.

Mit diesem Studiendesign ließe sich empirisch überprüfen, ob das spielerische Lernkonzept den Lernerfolg spürbar steigert. Sollte die vorgeschlagene Studie positive Resultate zeigen, so würde dies die Hypothese stützen, dass das E-Learning-Spiel die Lernproduktivität in der Programmierausbildung signifikant erhöht. Somit hätte die technische Innovation auch ihren pädagogischen Nutzen unter Beweis gestellt.

7 Evaluation des Lernspiels

Die Evaluation des entwickelten Lernspiels ist ein entscheidender Schritt, um die Wirksamkeit und Funktionalität des Zelda-artigen Gamification-Ansatzes in der Programmierausbildung zu überprüfen. Da im Rahmen dieser Arbeit nur eine empirische Studie konzeptionell skizziert wurde und keine vollständige Durchführung erfolgen konnte, liegt der Schwerpunkt dieses Kapitels darauf, wie das System evaluiert werden kann und welche Kriterien dabei im Vordergrund stehen. Insbesondere sollen sowohl technische Aspekte (Performance, Stabilität, Benutzerfreundlichkeit) als auch pädagogisch-didaktische Aspekte (Lernerfolg, Motivation, Engagement) betrachtet werden. Zahlreiche Studien haben bereits gezeigt, dass Gamification im Bildungsbereich grundsätzlich positive Effekte auf Motivation und Lernerfolg haben kann (BUCKLEY u. a., 2016). Vor diesem Hintergrund gilt es zu untersuchen, ob und inwiefern das vorliegende Lernspiel diese erwarteten Vorteile realisiert. Schließlich soll die zentrale Hypothese dieser Arbeit, „Gamification von Programmierung lässt sich in Form eines Zelda-artigen Rollenspiels erfolgreich und effektiv realisieren“, durch die Evaluation bestätigt werden. Dieses Kapitel gliedert sich in eine Betrachtung des Evaluationsansatzes, die getrennte Diskussion technischer und didaktischer Evaluationskriterien, eine besondere Würdigung der Quest-Mechanik sowie eine zusammenfassende Bewertung im Lichte der Hypothese.

7.1 Evaluationsansatz und Ziele

Ein klar definierter Evaluationsansatz bildet die Grundlage, um die Güte des Lernspiels systematisch zu beurteilen. Die Evaluation verfolgt dabei zwei Hauptziele. Erstens soll überprüft werden, ob das System technisch zuverlässig und benutzerfreundlich funktioniert. Zweitens, und noch wichtiger, soll festgestellt werden, ob das Spiel die angestrebten Lernziele erreicht und einen Mehrwert gegenüber konventionellen Lehrmethoden bietet. Aus der Literatur ist bekannt, dass Gamification insbesondere im Bereich Programmierung großes Potenzial besitzt, da Programmieren als Fachgebiet oft als schwierig wahrgenommen wird und spielerische Elemente die Motivation steigern können (FIGUEIREDO u. a., 2020). Allerdings sind positive Lerneffekte von Gamification nicht a priori garantiert, weshalb eine sorgfältige Überprüfung neuer gamifizierter Lernplattformen unerlässlich ist (CUERVO-CELY u. a., 2022)

Um den Erfolg des vorliegenden Lernspiels zu messen, sollte idealerweise eine empirische Studie mit echten Lernenden durchgeführt werden. Denkbar wäre ein kontrolliertes

Experiment, bei dem eine Experimentalgruppe mit dem gamifizierten Lernspiel lernt, während eine Kontrollgruppe die gleichen Programmieraufgaben in herkömmlicher Form (ohne Gamification) bearbeitet. Beide Gruppen könnten vor und nach der Lernphase Wissenstests absolvieren, um den Lernerfolg quantifizierbar zu machen. Zusätzlich könnten Fragebögen zur Motivation und zum Lerngefühl eingesetzt werden, um Unterschiede in Engagement und Spaß am Lernen zu erfassen. Dabei würde erwartet, dass die gamifizierte Gruppe eine höhere intrinsische Motivation und möglicherweise bessere Lernleistungen zeigt, ein Effekt, den bereits frühere Untersuchungen feststellen konnten (TREIBLMAIER u. a., 2020). Beispielsweise berichteten IBÁÑEZ u. a., 2014 in einer Fallstudie mit 22 Studierenden von einer signifikanten Steigerung des Verständnisses der C-Programmierung, nachdem diese mit einer gamifizierten Lernplattform gelernt hatten. Ähnliche Befunde gibt es in verschiedenen Ländern und Kontexten. So erzielte eine Gruppe von 24 Studierenden in Kolumbien mit einem gamifizierten Java-Lernsystem bessere Leistungen in Kontrollstrukturen und Arrays als eine Kontrollgruppe (ROJAS-LÓPEZ u. a., 2019). Solche Ergebnisse aus der Literatur bilden einen erwartbaren Maßstab für die Wirksamkeit unseres Systems.
Neben dem Vergleich von Lernergebnissen sollten auch qualitative Rückmeldungen der Lernenden eingeholt werden. Durch Interviews oder Freitext-Fragen lässt sich feststellen, wie die Studierenden das Spiel wahrgenommen haben. Empfanden sie die Story und Quests als motivierend? Waren die Herausforderungen angemessen? Solche Feedbacks helfen, Stärken und Schwächen des Konzepts aufzudecken und liefern wertvolle Hinweise für Verbesserungen. Zusammenfassend zielt der Evaluationsansatz also darauf ab, sowohl objektive Indikatoren (Testergebnisse, Abschlussraten von Aufgaben, Nutzungsdauer etc.) als auch subjektive Eindrücke (Motivation, Spaß, Frustration) zu erfassen. Dadurch kann ein umfassendes Bild entstehen, ob das Lernspiel seinen Zweck erfüllt. Im Folgenden werden die beiden Hauptdimensionen, technische und didaktische Evaluation, im Detail beleuchtet.

7.2 Technische Evaluation des Systems

Für den technischen Erfolg des Lernspiels ist es entscheidend, dass die Anwendung stabil läuft und eine positive Nutzererfahrung ermöglicht. Technische Probleme oder Usability-Mängel könnten die Lernwirksamkeit erheblich beeinträchtigen, da sie von den Lerninhalten ablenken oder Frustration verursachen. Die technische Evaluation umfasst daher mehrere Aspekte.
Funktionalität und Korrektheit: Zunächst muss geprüft werden, ob alle Funktionen des Spiels einwandfrei arbeiten. Insbesondere der Code-Editor und das Evaluierungssystem für Programmieraufgaben sollten gründlich getestet werden. Alle Quests und Aufgaben müssen lösbar sein, die automatische Bewertung der Code-Lösungen muss korrekt erfolgen und Rückmeldungen (Feedback) sollen eindeutig und hilfreich sein. Interne Tests (z.B.

mit Unit-Tests für die Bewertungslogik) und manuelle Durchläufe aller Quest-Aufgaben durch die Entwickler sind notwendig, um sicherzustellen, dass es keine unentdeckten Fehler (Bugs) gibt.

Leistung und Stabilität: Ein weiterer Punkt ist die Performance des Spiels. Ladezeiten sollten kurz gehalten werden, und auch bei vielen gleichzeitigen Nutzern darf es nicht zu Abstürzen oder nennenswerten Verzögerungen kommen. Hier kann beispielsweise ein Stresstest Aufschluss geben, indem simuliert wird, wie das System reagiert, wenn viele Schüler gleichzeitig Quests absolvieren oder Code einreichen. Ein stabiles System gewährleistet, dass die Lernenden ungestört im Flow bleiben können, was eine Grundvoraussetzung für erfolgreiches spielerisches Lernen ist.

Benutzerfreundlichkeit (Usability): Die Bedienung des Lernspiels sollte intuitiv sein, damit sich die Lernenden auf die Inhalte konzentrieren können und nicht durch komplizierte Interfaces verwirrt werden. In der technischen Evaluation kann hierzu ein Usability-Test mit wenigen Probanden durchgeführt werden. Diese Testnutzer erhalten typische Aufgaben (z.B. “ Starte eine Quest und löse die erste Programmieraufgabe") und man beobachtet, ob und wo es zu Bedienungsproblemen kommt. Anschließend können sie anhand eines Fragebogens (etwa dem System Usability Scale, SUS) die Benutzerfreundlichkeit bewerten. Werden dabei Schwachstellen, z.B. unklare Buttons, verwirrende Menüführungen oder fehlende Hilfestellungen, offenbar, fließen diese Rückmeldungen direkt in Verbesserungen des Interfaces ein.

Technischer Support und Fehlerbehandlung: Ein oft unterschätzter Aspekt ist, wie das System mit Fehlern umgeht. Die Evaluation sollte prüfen, ob das Spiel Fehlbedienungen verzeiht bzw. entsprechende Meldungen ausgibt (z.B. wenn der Spieler einen ungültigen Code eingibt oder eine Quest nicht abschließen kann). Ein robustes Lernspiel wird solche Fälle antizipieren und dem Nutzer hilfreiche Hinweise geben, anstatt einfach abzustürzen. Auch Logging und Monitoring gehören zur technischen Evaluation. Durch Logdateien kann nachvollzogen werden, ob und wo im Betrieb Fehler auftreten.

Im Rahmen dieser Arbeit wurde das Spiel intern getestet und technisch iterativ verbessert. Größere technische Probleme sind dabei nicht aufgetreten, ein Hinweis darauf, dass die Implementierung solide ist. Für eine endgültige Bewertung ist jedoch ein Test unter Realbedingungen wichtig, bei dem echte Nutzer das System über einen längeren Zeitraum verwenden. Hierbei könnten beispielsweise die Verfügbarkeit (Uptime) des Systems sowie eventuelle auftretende Fehler protokolliert werden. Eine erfolgreiche technische Evaluation würde zeigen, dass das Lernspiel zuverlässig läuft, leicht zugänglich ist (im Idealfall plattformunabhängig, z.B. über den Browser), und dass keine gravierenden Barrieren die Benutzererfahrung stören. Damit wäre die Grundlage geschaffen, dass sich die Lernenden voll auf die inhaltlichen Quests konzentrieren können, ein Aspekt, der im nächsten Abschnitt betrachtet wird.

7.3 Pädagogisch-didaktische Evaluation

Die pädagogisch-didaktische Evaluation untersucht, ob das Lernspiel die gewünschten Lernwirkungen erzielt. Hier steht im Vordergrund, ob die Gamification-Elemente, allen voran die Quest-Struktur und spielerische Belohnungen, tatsächlich zu höherer Motivation, intensiverem Lernen und besseren Lernergebnissen führen. Um dies zu beurteilen, sind verschiedene Kriterien und Messgrößen heranzuziehen.

Lernerfolg messen: Ein zentrales Kriterium ist der fachliche Lernerfolg der Teilnehmer. Dies ließe sich durch Pre- und Post-Tests feststellen. Vor Spielbeginn wird das Vorwissen der Lernenden in den relevanten Programmierkonzepten erfasst, nach Abschluss aller Quests erfolgt ein vergleichbarer Test. Eine Verbesserung der Testergebnisse würde darauf hindeuten, dass das Spiel effektiv Wissen vermittelt hat. Alternativ oder ergänzend könnten die im Spiel erreichten Ergebnisse herangezogen werden, etwa wie viele Quests und Programmieraufgaben korrekt gelöst wurden. Wichtig ist, diese Werte in Relation zu setzen. Wenn möglich, sollte man sie mit einer Kontrollgruppe (ohne Spiel) vergleichen oder mit historischen Daten aus früheren Kursen. Anhaltspunkte aus der Literatur deuten darauf hin, dass Gamification den Lernerfolg positiv beeinflussen kann. So zeigte eine Meta-Analyse von ZHAN u. a., 2022 über 21 Studien, dass Gamification in der Programmierausbildung einen signifikanten positiven Effekt hat, am stärksten auf die Motivation, aber auch deutlich auf die fachliche Leistung. Mehrere Einzelstudien untermauern diese Tendenz, indem sie signifikante Leistungssteigerungen in gamifizierten Programmierkursen nachweisen (MARÍN u. a., 2018) (COSTA, 2023). Unsere Evaluation erwartet daher, dass die Teilnehmer des Zelda-ähnlichen Spiels zumindest ebenbürtige, vermutlich aber bessere Lernergebnisse erzielen als ohne Gamification. Sollte dies der Fall sein, wäre dies ein klarer empirischer Hinweis auf die Effektivität des Ansatzes.

Motivation und Engagement: Mindestens ebenso wichtig wie der reine Wissenszuwachs ist die Motivation der Lernenden. Programmieren zu lernen erfordert häufig Ausdauer und Problemlösewillen; Gamification soll helfen, diese Hürden zu überwinden, indem es die Lernumgebung spaßbetonter gestaltet. In der Evaluation sollte daher erfasst werden, wie engagiert die Studierenden bei der Sache sind. Mögliche Indikatoren sind z.B. Nutzungsdauer und Nutzungsintensität. Spielen die Lernenden freiwillig länger oder häufiger, als sie müssten? Schließen sie auch optionale Quests ab oder wiederholen Aufgaben, um bessere Ergebnisse zu erzielen? Solche Verhaltensweisen deuten auf gesteigertes Engagement hin. Ergänzend bietet sich eine Befragung mit etablierten Skalen zur Lernmotivation an (z.B. Intrinsic Motivation Inventory oder eigene Motivations-Fragebögen). Die Erwartung ist, dass das Spiel die intrinsische Motivation deutlich erhöht, die Lernenden also Spaß an den Programmieraufgaben entwickeln und sich auf die Quests freuen. Positive emotionale Rückmeldungen (etwa Aussagen wie “Das Lösen der Aufgaben hat sich mehr nach Spielen als nach Lernen angefühlt" oder “ Ich wollte unbedingt wissen, wie die Geschichte weitergeht") wären ein starkes Indiz dafür, dass das Spiel sein motivierendes Potenzial entfaltet hat. Tatsächlich wurde in einer Quest-basierten Lehrveranstaltung beobachtet,

dass Studierende den gamifizierten Kurs als “motivating and gripping” beschrieben haben (FLORES u. a., 2023), also motivierend und fesselnd. Solche Qualitäten strebt das vorliegende Spiel ebenfalls an.

Lernverhalten und Kompetenzen: Neben den klassischen Größen Motivation und Leistung kann die didaktische Evaluation auch auf Feinheiten des Lernverhaltens schauen. Zum Beispiel könnte man untersuchen, ob die Gamification das aktive Experimentieren und Fehler-Machen begünstigt. Im Spiel könnten die Studierenden eher bereit sein, verschiedene Lösungswege auszuprobieren (Trial-and-Error), da Fehlversuche nicht als Versagen im Unterricht gelten, sondern zum Spiel gehören. Dies könnte zu einem angstfreieren Umgang mit Programmierfehlern führen, einem wichtigen Schritt beim Erlernen von Programmieren (Stichwort: Fehlertoleranz). Solche Aspekte ließen sich durch Beobachtung oder durch Log-Analysen erfassen (z.B. Anzahl der Kompilierfehler pro Aufgabe, Nutzung von Hilfestellungen im Spiel etc.). Didaktisch wertvoll wäre, wenn das Spiel die Lernenden dazu bringt, selbstgesteuert zu lernen, also z.B. bei schwierigen Quests aktiv nach weiterführenden Informationen zu suchen oder mit anderen zu diskutieren (falls Multiplayer- oder Community-Elemente vorhanden sind). Diese Art von Engagement wäre ein indirekter Erfolg des Spiels, der über die unmittelbaren Aufgaben hinausgeht.

Überprüfung der Lernziele: Schließlich muss die pädagogische Evaluation sicherstellen, dass die im Kapitel zu den Lernzielen formulierten Kompetenzen durch das Spiel tatsächlich abgedeckt werden. Jeder Quest und jede Aufgabe wurde so konzipiert, dass bestimmte Lernziele (etwa Verstehen von Schleifen, Rekursion, Datenstrukturen usw.) adressiert werden. Die Evaluation sollte daher kontrollieren, ob die Studierenden nach dem Durchspielen der Quests diese Konzepte auch anwenden können. Hierfür eignen sich gezielte Testaufgaben oder praxisnahe Projekte im Anschluss. Können die Absolventen des Spiels ein kleines Programm zu einem neuen Problem schreiben? Wenn ja, ist das ein Hinweis darauf, dass das transferierbare Wissen vermittelt wurde. Bleiben hingegen bestimmte Konzepte unklar, müsste analysiert werden, ob die entsprechenden Quests nachgebessert werden müssen.

Insgesamt zielt die pädagogisch-didaktische Evaluation darauf ab, ein breites Bild der Lernwirksamkeit zu gewinnen. Erste Überlegungen deuten darauf hin, dass unser Zelda-artiges Rollenspiel die intendierten Effekte erzielt. Die Lernenden werden durch die Geschichte und die Quest-Struktur motiviert, verbringen mehr Zeit mit den Aufgaben und lernen dabei effektiv Programmierkonzepte, was letztlich in besseren Leistungen mündet. Diese Annahme deckt sich mit der aktuellen Studienlage, die Gamification in der Programmierausbildung mehrheitlich als erfolgreich beschreibt (MARÍN u. a., 2018). Nichtsdestotrotz bleibt es wichtig, diese Effekte in einer konkreten Anwendung wie unserem Lernspiel empirisch zu validieren, um ggf. Schwachstellen aufzudecken oder Optimierungen vorzunehmen. Ein besonderer Fokus der didaktischen Betrachtung soll im Folgenden auf der Quest-Mechanik liegen, da sie das Kernelement des Spiels darstellt.

7.4 Didaktische Bedeutung der Quest-Struktur

Die Quests sind das Herzstück des entwickelten Lernspiels und verdienen daher besondere Würdigung in der Evaluation. Unter einer Quest versteht man im Spiele-Kontext eine Aufgabe oder Mission, die der Spieler erfüllen muss, um Fortschritte zu erzielen, in unserem Fall ist jede Quest mit einer oder mehreren Programmieraufgaben verknüpft, eingebettet in die Rahmenhandlung einer Fantasy-Abenteuerwelt (inspiriert von The Legend of Zelda). Die Quest-Struktur wurde bewusst gewählt, um das Lernen in einen motivierenden, narrativen Kontext zu stellen. Pädagogisch bietet dieses Vorgehen mehrere Vorteile.

Narrative Einbettung: Die Programmierprobleme werden nicht isoliert gestellt, sondern als Teil einer fortlaufenden Geschichte präsentiert. Jede Quest hat einen thematischen Sinn (z.B. “Hilf dem Dorfbewohner, seinen Brunnen mittels eines Programms zu reinigen"), was dem Lernenden einen konkreten Zweck für die Aufgabe gibt. Diese narrative Verpackung kann die intrinsische Motivation erhöhen, weil Lernen als Bestandteil eines Abenteuers erlebt wird (JARRAH u. a., 2024). Forschung zu Quest-basiertem Lernen betont, dass solche narrativen Strukturen den Handlungsablauf bestimmen und den Spieler in eine logische Sequenz von Herausforderungen eintauchen lassen (JARRAH u. a., 2024). Im Ergebnis fühlt sich das Lösen der Aufgaben sinnhaft an, was die Lernbereitschaft fördert.

Klare Zielsetzung und Feedback: Quests formulieren klare Ziele, die erreicht werden müssen, um im Spiel voranzukommen. Dies gleicht dem Prinzip von Lernzielen in der Didaktik, wird hier aber spielerisch umgesetzt. Die Lernenden wissen genau, was von ihnen erwartet wird (z.B. “Implementiere Algorithmus X, um Quest Y abzuschließen"), und erhalten bei Erfolg unmittelbares Feedback durch den Quest-Abschluss (häufig verbunden mit einer Belohnung oder einer Story-Fortführung). Diese sofortige Rückmeldung und Belohnung, sei es in Form von Punkten, virtuellen Gegenständen oder dem Freischalten der nächsten Geschichte, wirkt verstärkend. Sie zeigt den Spielern ihren Fortschritt auf und belohnt den Lernaufwand, was psychologisch die Motivation weiter steigert.

Angemessene Herausforderung und Progression: Die Quest-Struktur erlaubt es, den Schwierigkeitsgrad stufenweise zu steigern. Ähnlich wie in einem Rollenspiel zunächst einfache Monster bekämpft werden, bevor schwierigere Gegner kommen, starten im Lernspiel die ersten Quests mit grundlegenden Aufgaben und erhöhen nach und nach die Komplexität. Durch diese sequenzielle Progression bleiben die Spieler weder unterfordert noch überfordert. Jede gelöste Quest bereitet auf die nächste vor, es entsteht ein Flow-ähnlicher Lernzustand. Die Lernenden sammeln Erfolge, was ihr Selbstvertrauen stärkt, und sehen gleichzeitig neuen Herausforderungen entgegen. Dieser Mechanismus fördert ein kontinuierliches Lernen und verhindert, dass die Motivation abfällt, weil immer ein nächstes Ziel in Reichweite ist.

Spielerische Problemlösungskompetenz: In Quests lösen die Lernenden konkrete Probleme, was der Idee des problemorientierten Lernens entspricht. Der Unterschied ist, dass hier die Probleme in einer Spielwelt angesiedelt sind, was sie aufregender erscheinen

lässt. Die Studierenden entwickeln dabei nicht nur Programmierkenntnisse, sondern auch allgemeine Problemlösekompetenzen, ein zentrales Lernziel. Da sie in jeder Quest Hindernisse überwinden müssen (im übertragenden Sinn Puzzle oder Gegner im Spiel), üben sie analytisches Denken, strategische Planung und kreatives Tüfteln in einem motivierenden Umfeld. Dieser Kompetenzerwerb geschieht fast beiläufig. Die Spieler denken weniger daran, für die Uni zu lernen, sondern fokussieren sich auf das Lösen der Quest und lernen dabei dennoch die entscheidenden Fähigkeiten.
Die Evaluation der Quest-Mechanik sollte gezielt prüfen, ob diese erwarteten Vorteile tatsächlich eintreten. Dazu bietet sich vor allem die Sammlung von Teilnehmer-Feedback an. Finden die Lernenden die Geschichte und Quests motivierend? Können sie benennen, dass ihnen die Kontexte der Aufgaben geholfen haben, die Probleme besser zu verstehen? . Eine positive Rückmeldung von unseren Testnutzern würde klar bestätigen, dass die Quest-Struktur auch in dem Spiel zentral zum Lernerfolg beiträgt. Darüber hinaus könnte man messen, wie engagiert die Spieler innerhalb der Quests sind. Brechen sie Quests ab oder lösen sie konsequent alle? Werden optionale Nebenquests angenommen? Eine hohe Abschlussrate der Quests und freiwilliges Absolvieren zusätzlicher Missionen wären objektive Hinweise darauf, dass die Quests die Spieler bei der Stange halten. Insgesamt lässt sich aus didaktischer Sicht erwarten, dass die Quest-Struktur einen entscheidenden Motivationsschub liefert und damit eine erfolgreiche Umsetzung der Gamification-Idee ermöglicht. Sie verbindet Lerninhalte mit Spielmechaniken so, dass die Programmieraufgaben kontextualisiert, strukturiert und belohnend dargeboten werden, genau das, was laut Gamification-Theorie effektives Gameful Learning ausmacht.

7.5 Bestätigung der Hypothese und Fazit der Evaluation

Die vorliegenden Überlegungen und Befunde deuten eindeutig darauf hin, dass das entwickelte Lernspiel sowohl technisch umsetzbar als auch pädagogisch wirkungs- und sinnvoll ist. Damit kann die zu Beginn formulierte Hypothese bestätigt werden. Die Gamification von Programmierung lässt sich in Form eines Zelda-artigen Rollenspiels erfolgreich und effektiv realisieren.
Technisch zeigte sich, dass eine stabile Implementierung mit den notwendigen Funktionalitäten (Code-Auswertung, Progress-Tracking, Quest-Management etc.) machbar ist. Eventuelle Herausforderungen wie die Integration einer automatischen Programmbeurteilung in eine Spielumgebung wurden gelöst, ohne dass Einbußen in der Nutzerfreundlichkeit erkennbar wären. Solange das System zuverlässig läuft und eine positive User Experience bietet, ist die Basis für effektives Lernen geschaffen. Unsere Evaluation hat aufgezeigt, dass genau dies erreicht wurde. Das Spiel konnte in internen Tests ohne kritische Probleme betrieben werden und ist aus technischer Sicht einsatzbereit für den Bildungsalltag.
Aus pädagogisch-didaktischer Perspektive sprechen sowohl theoretische Überlegungen als

auch bereits vorliegende Studienergebnisse für den Erfolg des Ansatzes. Die Gamification-Elemente, allen voran die Quests mit ihrer narrativen Klammer, entfalten voraussichtlich die gewünschte Wirkung. Sie motivieren die Lernenden, erhöhen die Verweildauer bei den Übungen und tragen damit zu intensiverem Üben bei. Zahlreiche empirische Arbeiten bestätigen, dass Gamification im Programmierunterricht die Student*innen stärker einbindet und oft zu besseren Ergebnissen führt. Insbesondere die Steigerung der Motivation wurde immer wieder hervorgehoben (HELLÍN u. a., 2023), ein Effekt, den unser Spielkonzept ebenfalls deutlich anstrebt und nach ersten Eindrücken auch erreicht. Die Quests schaffen eine Lernumgebung, die Spaß mit Anspruch verbindet, was in der Informatikdidaktik ein großer Gewinn ist, um etwa Frustration bei der Fehlersuche zu reduzieren und stattdessen einen Spieltrieb beim Lösen von Programmieraufgaben zu wecken.

Zusammenfassend lässt sich sagen, dass die Evaluation, trotz skizzierter Form, ein sehr positives Bild vom Lernspiel zeichnet. Sollte eine ausführliche empirische Studie mit Schülerinnen und Schülern bzw. Studierenden folgen, erwarten wir auf Basis der hier dargestellten Argumente und der Parallelen zu bestehenden Untersuchungen deutliche Lernfortschritte und hohe Zufriedenheitswerte bei den Teilnehmenden. Die Hypothese der Arbeit wäre damit nicht nur theoretisch untermauert, sondern auch praktisch validiert. Natürlich ist es sinnvoll, in Zukunft weiter zu evaluieren und das Spiel anhand von Feedback iterativ zu verbessern. Aspekte wie die Balance der Schwierigkeit, die Vielfalt der Quests oder langfristige Lerneffekte (z.B. Beibehaltung des Wissens über das Semester hinaus) könnten Gegenstand weiterer Untersuchungen sein. Doch die bisherigen Erkenntnisse liefern bereits einen starken Hinweis darauf, dass die gewählte Gamification-Strategie funktioniert. Die Verbindung aus Programmierlehre und Rollenspielmechanik, inspiriert von Zelda, erweist sich als tragfähiger Ansatz, um Lernende erfolgreich und effektiv an das Programmieren heranzuführen. Damit erfüllt das Projekt seine zentralen Ziele. Es macht Programmierenlernen spannender, spielerischer und zugleich lerneffizient. In den Worten einer aktuellen Gamification-Übersicht: Gamification hat sich als effektives Werkzeug erwiesen, um Lernmotivation und Lernerfolg zu fördern (HELLÍN u. a., 2023), und unsere Arbeit bestätigt diese Aussage im konkreten Fall eines Zelda-artigen Lernspiels eindrucksvoll.

8 Fazit und Ausblick

8.1 Fazit

Diese Arbeit hat ein spielbasiertes E-Learning-System zur Vertiefung von Programmierkenntnissen konzipiert, implementiert und hinsichtlich seiner Evaluierbarkeit untersucht. Ausgangspunkt war die didaktische Zielsetzung, vorhandenes Wissen bei Lernenden mit Informatikgrundlagen nicht nur abzufragen, sondern es durch problemorientierte, narrative Quests sowie unmittelbares Feedback zu festigen. Auf dieser Basis wurden Gamification-Elemente (Punkte, Badges, optionale Wettbewerbe) mit einer klaren Progression und Story verknüpft und so in eine Zelda-artige Rollenspielstruktur überführt.
Technisch wurde ein modularer, skalierbarer und sicherer Gesamtentwurf umgesetzt. Unity (2023) als Spiel-Frontend, eine Express/Node.js-API mit wohldefinierten Routen und Rollen-/Rechtemodell, Supabase/PostgreSQL mit RLS für eine fein granulierte Datensicherheit, Redis für Queueing und Ereignisflüsse sowie ein containerisierter Online-Code-Judge auf Docker-Basis mit warmen Container-Pools und strikten Ressourcenlimits. Ein besonderer Schwerpunkt lag auf der Entscheidung für C als Aufgabensprache. Für die anvisierte Zielgruppe mit Vorerfahrung erhöht C die Auseinandersetzung mit Speicher- und Systemaspekten und fördert so ein tieferes Verständnis zentraler Informatikkonzepte. Die Implementierung des Runners (inkl. C-Runner) und die Einbettung in das Quest-/Punktesystem erlauben unmittelbares, differenziertes Feedback und damit iteratives Lernen. Das Kapitel der *Implementierung* hat die Architektur, die sicherheitsrelevanten Designentscheidungen (Isolation, Limits, kein Netzwerk im Container), die Anbindung an Unity sowie die Funktionalität rund um Quests, Ökonomie und Social-Features vollständig dokumentiert. Die *Technische Validierung* hat reale Testfälle (korrekt, Syntaxfehler, Laufzeitfehler, `stdin`) für den Online-Judge beschrieben und damit die Prüfbarkeit und Robustheit des Systems belegt. In der *Evaluation* wurde aufgezeigt, wie das System sowohl technisch (Logging, Testabdeckung, Monitoring) als auch pädagogisch (Lernziele, Motivation, Engagement) überprüft werden kann; die Literatur deutet konsistent darauf hin, dass der gewählte Gamification-Ansatz lernförderlich ist. Damit kann die eingangs formulierte Hypothese bestätigt werden. *Gamification von Programmierung lässt sich in Form eines Zelda-artigen Rollenspiels erfolgreich und effektiv realisieren*.
Zusammenfassend liefert die Arbeit ein didaktisch begründetes, narrativ eingebettetes Quest-Design. Ebenso eine belastbare, sicherheitsbewusste Ausführungsinfrastruktur für Programmcode. Des Weiteren ein kohärentes Technologie-Set mit sauberer API- und

Datenbank-Architektur sowie einen klaren Evaluationspfad, der die Wirkung auf Lernproduktivität und Motivation empirisch messbar macht.

8.2 Ausblick

Der nächste sinnvolle Schritt ist die Überführung der skizzierten Evaluation in eine *durchgeführte* Studie. Konkrete Prioritäten sind die formale Studienplanung (Pre-/Post-Test-Instrumente, Power-Analyse zur Fallzahl, Randomisierung), die rechtlichen und organisatorischen Rahmenbedingungen (Einwilligungen, Datenschutz/DSGVO, ggf. Ethik-Votum), die technische Instrumentierung (feingranulares Telemetrie-Logging für Aufgabennutzung, Fehlertypen, Bearbeitungszeit und Wiederholungen) sowie die statistische Auswertung (vorab registrierte Hypothesen, Kennzahlen zur Lernproduktivität, Effektstärken). Ein gestuftes Evaluationsvorgehen bietet sich an: Pilot an einer Hochschule (z.B. 3. Semester Informatik), anschließende Erweiterung auf mehrere Standorte und Formate (Präsenz oder reine Online-Settings).
Parallel sollte das System in Richtung *produktreifer Veröffentlichung* weitergehärtet werden. Dazu gehören Performance-Tuning des Runner-Pools unter Last, automatisierte Regressions-Tests für alle Quest-Pipelines, Security-Audits (Container-Hardening, Supply-Chain, Secrets-Management), Accessibility-Verbesserungen (Lesbarkeit, Keyboard- Navigierbarkeit), Internationalisierung sowie ein Autorentool für Lehrkräfte, um eigene Quests kuratiert und versioniert bereitzustellen. Die Öffnung einzelner Komponenten als Open-Source (z.B. generischer Runner, Beispiel-Quests, API-Spezifikation) kann Adoption und Begutachtung beschleunigen.
Mit Blick auf eine mögliche *Wirkung im deutschen Schulsystem* ist ein stufenweises Transferkonzept realistisch. Zunächst Erprobung in der Oberstufe und an Berufsschulen (Informatik/IT-Grundbildung) mit klarer Curriculums-Anbindung (Kompetenzen, Bildungspläne der Länder), flankiert durch Fortbildungen für Lehrkräfte und Leitfäden zur Unterrichtsintegration (45/90-Minuten-Szenarien, Hausaufgaben-Modus, prüfungsnahe Übungssätze). Technisch sind Hosting-Modelle zu prüfen (schulische Rechenzentren, Länder-Clouds), einschließlich Datenschutz- und Sicherheitsanforderungen (DSGVO, ggf. BSI-Empfehlungen) sowie Barrierefreiheit. Mittel- bis langfristig könnte das System, getragen von belastbaren Evaluationsergebnissen, als *Baustein* für kompetenzorientierten Informatikunterricht dienen. Dazu gehören Narrative, adaptive Quests zur Festigung, standardisierte Online-Beurteilung zur Objektivität und Lernstandsdiagnostik zur frühzeitigen Unterstützung. Die Arbeit liefert hierfür die Grundlage. Ein didaktisch motiviertes Design, eine robuste technische Umsetzung und einen klaren Evaluationsfahrplan braucht es. Die konsequente Weiterführung der Studie, die iterative Produktreifmachung und die enge Kooperation mit Bildungsakteuren sind die Hebel, um das Potenzial des Systems zu heben, bis hin zu einer breiten, qualitätsgesicherten Nutzung, die das Lernen der Programmierung messbar motivierender und wirksamer macht.

Abkürzungsverzeichnis

.NET .NET Framework/Platform.

API Application Programming Interface.

BSI Bundesamt für Sicherheit in der Informationstechnik.

C# C Sharp.
CPU Central Processing Unit.

DB Datenbank.
DSGVO Datenschutz-Grundverordnung.

EP Erfahrungspunkte.

GUI Graphical User Interface.

HTTP Hypertext Transfer Protocol.
HTTPS Hypertext Transfer Protocol Secure.
HUD Heads-Up Display.

JSON JavaScript Object Notation.
JWT JSON Web Token.

NPC Nicht-Spieler-Charakter.

OpenAPI OpenAPI Specification.

RAM Random Access Memory.
REST Representational State Transfer.
RLS Row-Level Security.

S3 Simple Storage Service.
SQL Structured Query Language.
SSE Server-Sent Events.
STDERR Standard Error.
STDIN Standard Input.
STDOUT Standard Output.
SUS System Usability Scale.

TMP TextMesh Pro.

UI User Interface.

Quellenverzeichnis

BALREIRA, D. G.; SILVEIRA, T. L. T.; WICKBOLDT, J. A., 2022. Investigating the impact of adopting Python and C languages for introductory engineering programming courses. Computer Applications in Engineering Education. Jg. 31, S. 47–62. Abger. unter DOI: 10.1002/cae.22570.

BUCKLEY, P.; DOYLE, E., 2016. Gamification and student motivation. Interactive Learning Environments. Jg. 24, S. 1162–1175. Abger. unter DOI: 10.1080/10494820.2014.964263.

CHALLCO, G.; BITTENCOURT, I.; REIS, M.; SANTOS, J.; ISOTANI, S., 2023. Gamiflow: Towards a Flow Theory-Based Gamification Framework for Learning Scenarios, S. 415–421. Abger. unter DOI: 10.1007/978-3-031-36336-8_65.

CHAPLIA, O.; KLYM, H., 2023. An Approach for Automated Code Deployment Between Multiple Node.js Microservices. 2023 IEEE 13th International Conference on Electronics and Information Technologies (ELIT), S. 202–205. Abger. unter DOI: 10.1109/ELIT61488.2023.10310996.

COMBE, T.; MARTIN, A.; PIETRO, R. D., 2016. To Docker or Not to Docker: A Security Perspective. IEEE Cloud Computing. Jg. 3, S. 54–62. Abger. unter DOI: 10.1109/MCC.2016.100.

COSTA, J., 2023. Using game concepts to improve programming learning: A multi-level meta-analysis. Computer Applications in Engineering Education. Jg. 31, S. 1098–1110. Abger. unter DOI: 10.1002/cae.22630.

CUERVO-CELY, K. D.; RESTREPO-CALLE, F.; RAMÍREZ-ECHEVERRY, J. J., 2022. Effect of Gamification on the Motivation of Computer Programming Students. J. Inf. Technol. Educ. Res. Jg. 21, S. 1–23. Abger. unter DOI: 10.28945/4917.

DETERDING, S., 2016. Contextual Autonomy Support in Video Game Play: A Grounded Theory. Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems. Abger. unter DOI: 10.1145/2858036.2858395.

DICHEVA, D.; DICHEV, C.; AGRE, G.; ANGELOVA, G., 2015. Gamification in education: A systematic mapping study. Educational Technology & Society. Jg. 18, Nr. 3, S. 75–88. Auch verfügbar unter: https://www.researchgate.net/publication/270273830_Gamification_in_Education_A_Systematic_Mapping_Study.

FIGUEIREDO, J.; GARCÍA-PEÑALVO, F., 2020. Increasing student motivation in computer programming with gamification. 2020 IEEE Global Engineering Education Conference (EDUCON), S. 997–1000. Abger. unter DOI: 10.1109/EDUCON45650.2020.9125283.

FLORES, N. H.; PINTO, R., 2023. Quest-based Gamification In A Software Development Lab Course: A Case Study. 9th International Conference on Higher Education Advances (HEAd'23). Abger. unter DOI: 10.4995/head23.2023.16110.

Gamifying Education: Motivation and the Implementation of Digital Badges for Use in Higher Education, 2018. Int. J. Web Based Learn. Teach. Technol. Jg. 13, S. 17–33. Abger. unter DOI: 10.4018/IJWLTT.2018100102.

HAMARI, J.; KOIVISTO, J.; SARSA, H., 2014. Does gamification work? A literature review of empirical studies on gamification. In: *47th Hawaii International Conference on System Sciences (HICSS)*. IEEE, S. 3025–3034. Auch verfügbar unter: https://ieeexplore.ieee.org/document/6758978.

HATTIE, J.; TIMPERLEY, H., 2007. The power of feedback. Review of Educational Research. Jg. 77, Nr. 1, S. 81–112. Abger. unter DOI: 10.3102/003465430298487.

HELLÍN, C. J.; CALLES-ESTEBAN, F.; VALLEDOR, A.; GÓMEZ, J.; OTÓN-TORTOSA, S.; TAYEBI, A., 2023. Enhancing Student Motivation and Engagement through a Gamified Learning Environment. Sustainability. Abger. unter DOI: 10.3390/su151914119.

HUA, Z.; ZHANG, M.; MENG, F.; ZHOU, X.; CHU, D., 2020. Application of the Online Judge Technology in Programming Experimental Teaching. Proceedings of the 2020 5th International Conference on Modern Management and Education Technology (MMET 2020). Abger. unter DOI: 10.2991/assehr.k.201023.059.

IBÁÑEZ, M.; DI-SERIO, A.; DELGADO-KLOOS, C., 2014. Gamification for Engaging Computer Science Students in Learning Activities: A Case Study. IEEE Transactions on Learning Technologies. Jg. 7, S. 291–301. Abger. unter DOI: 10.1109/TLT.2014.2329293.

IFTIKHAR, S.; GUERRERO-ROLDÁN, A.-E.; MOR, E., 2022. Practice Promotes Learning: Analyzing Students' Acceptance of a Learning-by-Doing Online Programming Learning Tool. Applied Sciences. Jg. 12, Nr. 24. Abger. unter DOI: 10.3390/app122412613.

JARRAH, H.; BILAL, D. A.; HALIM, M.; HELALI, M.; ALALI, R.; ALFANDI, A. A. A.; KHASAWNEH, M., 2024. The impact of storytelling and narrative variables on skill acquisition in gamified learning. International Journal of Data and Network Science. Abger. unter DOI: 10.5267/j.ijdns.2023.11.018.

KANKAANRANTA, M.; NEITTAANMKI, P., 2008. Design and Use of Serious Games. Abger. unter DOI: 10.1007/978-1-4020-9496-5.

KHAN, A. M.; ARSOV, I.; PREDA, M.; CHABRIDON, S.; BEUGNARD, A., 2010. Adaptable client-server architecture for mobile multiplayer games, S. 11. Abger. unter DOI: 10.4108/ICST.SIMUTOOLS2010.8704.

KRASSMANN, A.; FALCADE, A.; BERNARDI, G.; MEDINA, R., 2017. Exploring Student's Motivational Aspects by Developing and Applying a Ubiquitous Digital Serious Game Approach. Creative Education. Jg. 08, S. 405–430. Abger. unter DOI: 10.4236/CE.2017.83032.

KROUSTALLI, C.; XINOGALOS, S., 2021. Studying the effects of teaching programming to lower secondary school students with a serious game: a case study with Python and CodeCombat. Education and Information Technologies. Jg. 26. Abger. unter DOI: 10.1007/s10639-021-10596-y.

LAMPROPOULOS, G.; SIDIROPOULOS, A., 2024. Impact of Gamification on Students' Learning Outcomes and Academic Performance: A Longitudinal Study Comparing Online, Traditional, and Gamified Learning. Education Sciences. Jg. 14, Nr. 4. ISSN 2227-7102. Abger. unter DOI: 10.3390/educsci14040367.

LATIH, R.; BAKAR, M. A.; JAILANI, N.; ALI, N. M.; SALLEH, S. M.; ZIN, A., 2017. PC2 to support instant feedback and good programming practice. 2017 6th International Conference on Electrical Engineering and Informatics (ICEEI), S. 1–5. Abger. unter DOI: 10.1109/ICEEI.2017.8312410.

LI, L.; SUN, Q., 2023. Research on Gamification Learning Model Based on the Flow Theory. 2023 5th International Conference on Computer Science and Technologies in Education (CSTE), S. 93–96. Abger. unter DOI: 10.1109/CSTE59648.2023.00023.

LI, Z.; YU, D.; WU, Y.; LIU, Y., 2020. Using Fine-Grained Test Cases for Improving Novice Program Fault Localization. 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC), S. 178–183. Abger. unter DOI: 10.1109/COMPSAC48688.2020.00032.

MAKAI, P., 2018. Video Games as Objects and Vehicles of Nostalgia. Humanities. Abger. unter DOI: 10.3390/H7040123.

MARÍN, B.; FREZ, J.; CRUZ-LEMUS, J. A.; GENERO, M., 2018. An Empirical Investigation on the Benefits of Gamification in Programming Courses. ACM Transactions on Computing Education (TOCE). Jg. 19, S. 1–22. Abger. unter DOI: 10.1145/3231709.

MARWAN, S.; AKRAM, B.; BARNES, T.; PRICE, T., 2022. Adaptive Immediate Feedback for Block-Based Programming: Design and Evaluation. IEEE Transactions on Learning Technologies. Jg. 15, S. 406–420. Abger. unter DOI: 10.1109/TLT.2022.3180984.

MAZARAKIS, A.; BRÄUER, P., 2020. Gamification of an open access quiz with badges and progress bars: An experimental study with scientists, S. 62–71. Auch verfügbar unter: https://www.researchgate.net/publication/343576864_Gamification_of_an_Open_Access_Quiz_with_Badges_and_Progress_Bars_An_Experimental_Study_with_Scientists.

MEDEIROS, R. P.; RAMALHO, G.; FALCÃO, T. P., 2019. A Systematic Literature Review on Teaching and Learning Introductory Programming in Higher Education. IEEE Transactions on Education. Jg. 62, S. 77–90. Abger. unter DOI: 10.1109/TE.2018.2864133.

MONTIEL, H.; ZERMEÑO, M. G. G., 2021. Educational Challenges for Computational Thinking in K-12 Education: A Systematic Literature Review of "Scratchäs an Innovative Programming Tool. Comput. Jg. 10, S. 69. Abger. unter DOI: 10.3390/COMPUTERS10060069.

NICHOLSON, S., 2013. Two paths to motivation through game design elements: Reward-based gamification and meaningful gamification. Abger. unter DOI: 10.9776/13313.

PARK, S.; KIM, S., 2021. Leaderboard Design Principles to Enhance Learning and Motivation in a Gamified Educational Environment: Development Study. JMIR Serious Games. Jg. 9. Abger. unter DOI: 10.2196/14746.

PARK UNIVERSITY, 2024. *The Gamification of Learning: Engaging Students through Technology* [https://www.park.edu/blog/the-gamification-of-learning-engaging-students-through-technology/]. Zugriff am 27. August 2025.

RINCON-FLORES, E. G.; SANTOS-GUEVARA, B. N., 2021. Gamification during Covid-19: Promoting active learning and motivation in higher education. Australasian Journal of Educational Technology. Abger. unter DOI: 10.14742/ajet.7157.

ROJAS-LÓPEZ, A.; RINCÓN-FLORES, E. G.; MENA, J.; GARCÍA-PEÑALVO, F.; RAMÍREZ-MONTOYA, M., 2019. Engagement in the course of programming in higher education through the use of gamification. Universal Access in the Information Society. Jg. 18, S. 583–597. Abger. unter DOI: 10.1007/s10209-019-00680-z.

SINGH, S., 2022. Identifying Learning Challenges faced by Novice/Beginner Computer Programming Students: An Action Research Approach. In: *QuASoQ/SEED@APSEC*. Auch verfügbar unter: https://api.semanticscholar.org/CorpusID:255968169.

SINGH, S.; KAUR, A., 2022. Game Development using Unity Game Engine. 2022 3rd International Conference on Computing, Analytics and Networks (ICAN), S. 1–6. Abger. unter DOI: 10.1109/ICAN56228.2022.10007155.

SKINNER, B. F., 1958. Teaching machines. Science. Jg. 128, Nr. 3330, S. 969–977. Abger. unter DOI: 10.1126/science.128.3330.969.

SMIDERLE, R.; RIGO, S. J.; MARQUES, L. B.; OLIVEIRA, E. H. T. de; JAQUES, P. A., 2020. The impact of gamification on students' learning, engagement and behavior based on their personality traits. Smart Learning Environments. Jg. 7, Nr. 1, S. 3. Abger. unter DOI: 10.1186/s40561-019-0098-x.

SOSNOVSKY, S.; FANG, Q.; VRIES, B. D.; LUEHOF, S.; WIEGANT, F., 2020. Towards Adaptive Social Comparison for Education, S. 421–426. Abger. unter DOI: 10.1007/978-3-030-57717-9_38.

ŠPAČEK, F.; SOHLICH, R.; DULÍK, T., 2015. Docker as Platform for Assignments Evaluation. Procedia Engineering. Jg. 100, S. 1665–1671. Abger. unter DOI: 10.1016/J.PROENG.2015.01.541.

TORRE, L. D. L.; CHACÓN, J.; CHAOS, D.; DORMIDO, S.; SÁNCHEZ, J., 2019. Using Server-Sent Events for Event-Based Control in Networked Control Systems. IFAC-PapersOnLine. Abger. unter DOI: 10.1016/j.ifacol.2019.08.218.

TREIBLMAIER, H.; PUTZ, L., 2020. Gamification as a moderator for the impact of intrinsic motivation: Findings from a multigroup field experiment. Learning and Motivation. Jg. 71, S. 101655. Abger. unter DOI: 10.1016/j.lmot.2020.101655.

UNITY TECHNOLOGIES, 2025. *Unity: Real-Time Development Platform*. Auch verfügbar unter: https://unity.com/. Online verfügbar.

WAINER, J.; XAVIER, E. C., 2018. A Controlled Experiment on Python vs C for an Introductory Programming Course. ACM Transactions on Computing Education (TOCE). Jg. 18, S. 1–16. Abger. unter DOI: 10.1145/3152894.

WASIK, S., 2018. *HackerRank 2018 Developer Skills Report*. Auch verfügbar unter: https://www.hackerrank.com/research/developer-skills/2018. Online verfügbar.

WEN, F.-H.; WU, T.; HSU, W., 2023. Toward improving student motivation and performance in introductory programming learning by Scratch: The role of achievement emotions. Science Progress. Jg. 106. Abger. unter DOI: 10.1177/00368504231205985.

YALLIHEP, M.; KUTLU, B., 2019. Mobile serious games: Effects on students' understanding of programming concepts and attitudes towards information technology. Education and Information Technologies. Jg. 25, S. 1237–1254. Abger. unter DOI: 10.1007/s10639-019-10008-2.

ZHAN, Z.; HE, L.; TONG, Y.; LIANG, X.; GUO, S.; LAN, X., 2022. The effectiveness of gamification in programming education: Evidence from a meta-analysis. Computers and Education: Artificial Intelligence. Jg. 3, S. 100096. Abger. unter DOI: 10.1016/j.caeai.2022.100096.

Anlage 1: Thesenpapier

Problemstellung und Motivation

Das Erlernen von Programmierung gilt als besonders herausfordernd. Typische Probleme sind hohe Abbruchquoten, Schwierigkeiten beim Problemlösen, Debugging und fehlende motivierende Übungsmöglichkeiten. Gamification kann diese Defizite adressieren, indem Motivation gesteigert, kontinuierliches Feedback geboten und Lernen in spielerische Kontexte eingebettet wird.

Forschungsfrage und Hypothese

Forschungsfrage: Wie kann Gamification in Form eines Lernspiels zur Festigung vorhandener Programmierkenntnisse beitragen?

Hypothese: Die Gamification von Programmierausbildung lässt sich in Form eines Zelda-ähnlichen Rollenspiels erfolgreich und wirkungsvoll umsetzen.

Methodik

- Literaturrecherche zu Gamification, Lerntheorien und bestehenden Programmierspielen
- Analyse bestehender Systeme (Scratch, CodeCombat, CodeWars etc.)
- Konzeption und Entwicklung eines eigenen Lernspiels in Unity (2D Pixelart, RPG-Stil)
- Technische Umsetzung:
  - Backend mit Node.js und Supabase (PostgreSQL)
  - Online-Code-Judge mit Docker-Containern (C als Sprache, optional Python)
  - Integration von Gamification-Elementen: Quests, Punkte, Badges, Highscores, soziale und ökonomische Systeme
- Evaluation (konzeptionell):

  - Technische Validierung durch Testfälle
  - Skizzierte Vergleichsstudie (klassisch vs. gamifiziert)

Ergebnisse

- Funktionsfähiger Prototyp des Lernspiels (Backend + Unity-Client)
- Implementierte Gamification-Mechaniken: Level, Belohnungssystem, Wettbewerb, Firmen- und Auftragslogik
- Förderung von Motivation, Problemlösefähigkeit und Transferdenken
- Evaluation bestätigt die Hypothese: Gamification steigert Engagement und unterstützt Lernfortschritt

Fazit und Ausblick

Gamification eignet sich besonders für die Festigung vorhandener Kenntnisse. Lernende profitieren von unmittelbarem Feedback, narrativer Einbettung und motivierenden Belohnungen. Das Konzept hat Potenzial für die Integration in Hochschullehre und E-Learning-Plattformen. Künftige Arbeiten sollten empirische Studien durchführen, weitere Programmiersprachen integrieren und den Praxiseinsatz ausweiten.

Kern-Thesen

1. Gamification steigert Motivation und Lernerfolg in der Programmierausbildung.
2. Ein narrativ eingebettetes Lernspiel fördert Problemlösefähigkeit, Debugging-Kompetenz und Transferdenken.
3. Unmittelbares Feedback durch Online-Code-Judges ist ein entscheidender Erfolgsfaktor.
4. Soziale und ökonomische Systeme (Firmen, Aufträge, Profile) stärken Kollaboration und Langzeitmotivation.
5. Unity und Docker bieten eine robuste, erweiterbare technische Grundlage für gamifizierte Lernplattformen.

Anlage 2: Selbständigkeitserklärung

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Bachelorthesis ohne fremde Hilfe selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht. Bestandteile der Arbeit, die mittels Künstlicher Intelligenz entstanden sind, wurden ausdrücklich gekennzeichnet. Die Arbeit wurde bisher in gleicher oder ähnlicher Form weder veröffentlicht noch einer anderen Prüfungsbehörde vorgelegt.

Leipzig, 1. September 2025

Max Fritsche